

GEORGE EMIL PALADE UNIVERSITY OF MEDICINE, PHARMACY, SCIENCE, AND TECHNOLOGY OF TÂRGU MUREȘ

FACULTY OF ENGINEERING AND INFORMATION TECHNOLOGY

DEPARTMENT: Electrical Engineering and Information Technology

STUDY PROGRAMME: Computer Science

BACHELOR’S THESIS

Premade Graph Explorer

League of Legends Player Relationships, Performance Metrics, and Graph Analytics Extensions

SUPERVISOR

Prof. d. habil. Iantovics Barna-László

GRADUATE

Tamási Krisztián-Erik

CLASS OF 2026

Abstract

This thesis presents a multi-component software system that constructs player-relationship graphs from real League of Legends match data and analyses them using graph-theoretic, statistical, and evolutionary methods. The system comprises a Riot API-based data-collection layer, a role-aware performance scoring pipeline (*opscore*/*feedscore*), a Rust-powered graph construction and analytics runtime, and a multi-page React/Vite frontend.

The research component compares two datasets (a Flex Queue corpus, *flexset*, and a Solo Queue corpus, *soloq*) with respect to assortativity, betweenness centrality, and associative core-periphery structure. The implementation includes Dijkstra, bidirectional, and exact A* pathfinding algorithms, parallelised Brandes betweenness centrality, and 3D graph visualisation. As a closing experiment, the Tribal NeuroSim evolutionary simulation seeds initial agent behaviour from cluster profiles extracted from the player graph.

Keywords: graph analysis, League of Legends, assortativity, betweenness centrality, Rust, React, neuroevolution.

Contents

I PremadeGraph System and Graph Analytics 6

1 Introduction 7

1.1 Problem Statement 7

1.2 Motivation 7

1.3 Objectives 8

1.4 Research Questions 8

1.5 Scope and Limitations 9

1.6 Structure of the Thesis 9

2 Literature Review 10

2.1 Network Research and Community Structure 10

2.2 Community Detection and Modularity 10

2.3 Assortativity 10

2.4 Player Performance and MOBA Analytics 11

2.5 Shortest Paths and Heuristic Search 11

2.6 Agent-Based Modelling 11

3 Data Collection and the Riot API 12

3.1 Logic of the Riot API Endpoints Used 12

3.2 Collector State Machine and Premade-Probe 12

3.3 Collection Focus and Queue Selection 13

3.4 Rate Limits and Reliability 14

3.5 Comparison of flexset and soloq Collection Strategies 14

4 System Architecture and Evolution 16

4.1 The Early Version . . . 16
4.2 The Current System . . . 16
4.3 Technology Stack Rationale . . . 16
4.4 Multi-Dataset Support . . . 17
4.5 Current Data State . . . 18
4.6 Persistence and the Relational Model . . . 18

5 Comparing the Two Datasets: flexset and soloq 21
5.1 Structural Differences Between the Two Datasets . . . 21
5.2 Summary . . . 22

6 The opscore Performance Metric 23
6.1 Metric Design . . . 23
6.1.1 The Role of a Custom Performance Metric . . . 23
6.1.2 Limitations of the Earlier Model . . . 23
6.2 The Local, Role-Aware opscore Model . . . 24
6.3 The Five Artifacts and Their Subcategories . . . 24
6.4 Mathematical Formalism . . . 24
6.4.1 The Role Ratio and the Base Curve . . . 24
6.4.2 Duration Reliability . . . 26
6.4.3 Artifact Aggregation and Boosting . . . 26
6.4.4 The Final opscore . . . 26
6.4.5 Exceptional Performances and Edge Cases . . . 26
6.5 The Role of opscore in the System . . . 27

7 Player Network Construction, Search, and Measurements 28
7.1 Graph Construction . . . 28
7.1.1 Edge Aggregation . . . 28
7.1.2 The Rust Graph State . . . 28
7.1.3 Filtering and the Research Graph . . . 28
7.1.4 The Graph Artifact Layer . . . 29
7.2 Pathfinding and Visualisation . . . 29
7.2.1 Search Algorithms . . . 29
7.2.2 Exact A* and the ALT Heuristic . . . 29

7.2.3 Pathfinder Lab . . . 30
7.2.4 The 3D Global Visualisation . . . 31
7.3 Assortativity . . . 34
7.3.1 Computational Definition . . . 34
7.3.2 Assortativity Results: flexset . . . 34
7.3.3 Assortativity Results: soloq . . . 35
7.3.4 Interpretation . . . 36
7.4 Betweenness Centrality . . . 37
7.4.1 The Brandes Algorithm . . . 37
7.4.2 Parallel Execution . . . 37
7.4.3 Result . . . 37

8 The Database Sub-Project: PostgreSQL Schema and C# Browser 40
8.1 Background . . . 40
8.2 The Formal Relational Schema . . . 40
8.3 The ASP.NET Core / C# Database Browser . . . 41
8.4 The Sub-Project's Place in the System . . . 41

II Tribal NeuroSim 42

9 Tribal NeuroSim: Foundations and Architecture 43
9.1 Objectives and Research Connection . . . 43
9.2 Agent-Based and Multi-Agent Model . . . 43
9.3 System Architecture . . . 43
9.4 FrameV1 Binary Protocol . . . 44

10 Tribal NeuroSim: Game Logic and Neural Decision-Making 46
10.1 Neural Decision-Making and Cluster Priors . . . 46
10.2 The 11 Input Signals and 7 Output Drives . . . 46
10.3 Fitness, Mutation, and Lineage Inheritance . . . 46

11 Tribal NeuroSim: Experimental Runs and Results 49
11.1 World Mechanics . . . 49
11.2 Summary of the Four Runs . . . 49

11.3 103-Seed Sweep and Winner Archetype Distribution 51
11.4 What the Results Show 52

12 Conclusion **53**

12.1 What the Thesis Presents 53
12.2 Main Results 53
12.3 The Role of Tribal NeuroSim 54
12.4 Retrospective: What Worked Well 54
12.5 Limitations and Open Questions 54
12.6 Closing Remarks 55
12.7 Where This Applies Elsewhere 55

Part I

PremadeGraph System and Graph Analytics

1. Introduction

1.1 Problem Statement

A recurring problem in competitive online games is that players form strong opinions about each other's performance based on partial, subjective, and short-term impressions. After a lost match, blame-shifting arrives quickly, but it is far rarer for players to actually examine the match data, understand what happened, how they themselves performed, and where that performance fits within a broader social network.

League of Legends is a particularly interesting setting for this, because it simultaneously offers extremely rich match data, recurring teammate and opponent relationships, and social patterns that can be represented and analysed as a graph. When a player repeatedly ends up in the same game as the same people over an extended period — as allies or as opponents — the resulting relationship network goes beyond the momentary statistics of any single match.

This observation is the starting point of the thesis. The goal: build a player network from real match history, analyse that network using a graph-theoretic perspective, and display the results on a visually interpretable frontend.

1.2 Motivation

The project has partly personal origins. After ranked matches, a recurring question was whether repeated negative encounters were coincidental or had some underlying structure — perhaps the same people being placed together again and again, without a native interface in the client for viewing this. That alone is not a scientific question, but it was enough as a starting point. If the Riot API provides detailed match data and those matches contain recurring player relationships, then a graph can in principle be constructed from them, and where there is a graph, there is something to analyse.

The project scope expanded gradually during development. What began as quick data visualisation grew into a dedicated data-collection layer, a normalised scoring pipeline, SQLite-based persistence, a Rust runtime, and a multi-page React frontend. The Python layer remained for data collection and compatibility. Every extension resolved a specific technical constraint rather than following a pre-planned roadmap, leading to further exploration of the existing data.

At one point the graph-analysis pipeline began producing cluster-profile exports, and the question arose whether those profiles could be used as inputs to an agent-based simulation. That

is how Tribal NeuroSim emerged: the clusters from the two collected datasets compete under identical simulation conditions within a civilisation simulation.

1.3 Objectives

The thesis documents the design, implementation, and evaluation of a multi-component system built on real match data. The system constructs a player-relationship graph from League of Legends match history: the graph arises from repeatedly playing together or against one another and is accompanied by role-aware performance metrics. Graph analysis covers clustering, core-periphery structure, assortativity, and Brandes betweenness centrality, all in a Rust-based, deterministic, and reproducible implementation.

The player network is examined from three perspectives. From a network perspective: core-periphery structure can be examined in the *flexset* dataset. From a metric perspective: the role-aware *opscore* and *feedscore* measures are computed from real match data. From a systems perspective: each component in the Python, SQLite, Node/Express, Rust, and React stack serves a specific purpose that the other layers cannot fulfil.

Every step in the pipeline is re-runnable with concrete inputs and outputs: from data collection through graph construction and clustering to simulation. The parallel analysis of the Flex Queue and SoloQ datasets makes this measurable — the same pipeline on two different inputs produces different results, and the difference becomes interpretable through the queue mechanics.

1.4 Research Questions

Five concrete questions guide the investigation:

RQ1. How can a real League of Legends match history be used to build a player network that captures not only co-play but also opponent relationships?

RQ2. How can player performance be scored in a way that accounts for the actual role played while remaining interpretable?

RQ3. To what extent is it justified to split the system into Python, SQLite, Node.js, Rust, and React components?

RQ4. Does the current graph show positive assortativity with respect to *opscore* and *feedscore*?

RQ5. In what ways do the relationship structures of *flexset* and *soloq* differ, and to what extent can the *flexset* graph be described as an associative core-periphery player graph?

1.5 Scope and Limitations

The project is a relatively large system, and it would be easy to present every existing idea as a finished result. The thesis tries to avoid this. What actually runs and can be verified appears as complete. What development plans list as future goals does not appear as a finished feature.

1.6 Structure of the Thesis

The chapters proceed from data collection toward graph analytics and simulation experiments.

The opening chapters establish the starting point: the problem statement, the literature background, and the Riot API-based data sources. This is followed by the system overview, then the *opscore* performance metric, the graph construction and clustering mechanics, the Rust runtime and analytics modules. The comparison of the two datasets (*flexset* and *soloq*) is the most concrete research result of the system. The final chapters cover the database sub-project and the Tribal NeuroSim simulation. This document is the condensed, lite version; the extended version [7] develops in separate chapters the topics only briefly touched on here.

Figure 1.1 shows one entry point of the current frontend: the match analysis and data collection interface.

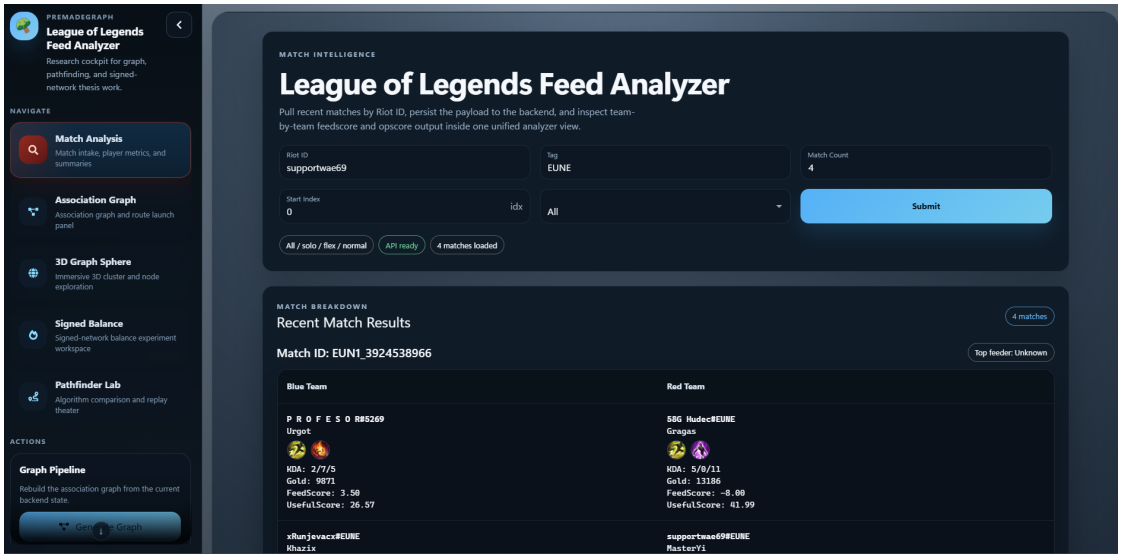


Figure 1.1: One entry point of the early frontend: the match analysis and data collection interface

2. Literature Review

2.1 Network Research and Community Structure

Graphs are a natural model for studying recurring interactions. For gaming communities in particular, a compelling question is to what extent the recurring interactions between players form recognisable structures, sub-communities, bridges, and local patterns. The classic work of Girvan and Newman demonstrated that network edges are not merely connective elements but relationships of varying importance that describe community structure [13]. Fortunato's comprehensive survey summarises why community detection is a central problem in modern network analysis [14].

Community structure is not an end in itself in this project. It matters because player groups built on repeatedly recurring connections make large networks more interpretable, help separate genuine relationship cores from noise, and provide support for subsequent pathfinding and analytic computations.

2.2 Community Detection and Modularity

The early Python-side population clustering used the NetworkX *greedy_modularity_communities* procedure, which is based on the Clauset–Newman–Moore greedy modularity maximisation approach [16, 6]. This was a deliberate trade-off: not the most impressive method, but one that is quickly explainable and reproducible. The current runtime graph state is built independently by the Rust engine from match JSON files and the SQLite database.

The known limitations of modularity optimisation cannot be ignored either: the result of Fortunato and Barthelemy shows that because of the resolution limit, smaller but genuine structures may merge in the interest of the global optimum [15]. The *python_population* clusters should therefore not be treated as ontological "true communities" but rather as the best reproducible population partition given the chosen graph, weight model, and heuristic.

2.3 Assortativity

Newman's work describes assortative mixing as a network phenomenon in which connected nodes carry similar or dissimilar properties [12]. This project applies that concept to player

performance metrics: the question is not whether high-degree nodes connect to each other, but whether players connected in the graph carry similar *opscore* or *feedscore* values.

The thesis distinguishes two graph modes: *social-path* builds only on positively supported ally connections, while *battle-path* includes all recurring co-play and head-to-head appearances. Assortativity differs between the two modes, and that difference numerically measures the behavioural distinction between the two graph modes.

2.4 Player Performance and MOBA Analytics

The esports analytics of League of Legends and MOBA games more broadly has grown into its own research area. The comprehensive literature review by Mora-Cantallops and Sicilia shows that MOBA games offer a unique combination of competition, cooperation, player experience, and social interaction [45].

Player performance modelling also appears in the work of Novak and colleagues: performance models developed for professional League of Legends matches indicate that domain-specific variables and structured data processing matter more than general statistical models [28]. This reinforces the rationale for the role-aware scoring approach in the current project.

Sánchez-González and González-Bailón connect the network structure of professional LoL teams to team efficiency [30].

2.5 Shortest Paths and Heuristic Search

The classical shortest-path problem was formalised by Dijkstra [17], while the formal basis of A* search was established by Hart, Nilsson, and Raphael [18]. The exact A* implementation in the Rust layer uses ALT-style lower bounds inspired by the Goldberg–Harrelson approach [19]. The goal is a correct and stable runtime suited to the scale of the player network — not a new algorithm, but a reliable application of existing, tested methods.

2.6 Agent-Based Modelling

Agent-based modelling (ABM) is a simulation paradigm in which individual actors behave according to local rules and macro-level patterns emerge as the aggregate of those decisions. Bonabeau and colleagues applied this framework to both natural and artificial systems [10].

The Tribal NeuroSim experiment forming the final part of the thesis fits into this paradigm. ABM literature emphasises that results obtained from a simulation should be interpreted as the behaviour of the model, not as a direct representation of reality.

3. Data Collection and the Riot API

Data collection is based on the Riot Developer Portal and the match-v5 API suite [1, 2]. These enable identifying player accounts, querying match lists, downloading detailed match JSON files, and processing participant statistics.

3.1 Logic of the Riot API Endpoints Used

The current project uses several layers of the Riot API, but the most important design decision was making *puuid* (a unique account identifier) the central identifier of the system. The pipeline breaks down logically into the following steps:

1. starting from an initial player or seed list, the system requests account information;
2. from the account, it retrieves a list of match identifiers;
3. for the match identifiers it downloads the detailed match JSON files;
4. from the participant rows it constructs a *puuid*-based player profile and relationship data;
5. the results are persisted bound to the dataset.

The collector calls the *match-v5* endpoint family via the *europe.api.riotgames.com* regional route, which provides access to match data.

3.2 Collector State Machine and Premade-Probe

backend/match_collector.py is a parameterisable state machine: its configuration holds the dataset identifier, paths, collection mode, queue type, match count limit, iteration limit, and premade threshold.

Figure 3.1 shows the full workflow of the collector's parameterisable state machine, from startup to persistence.

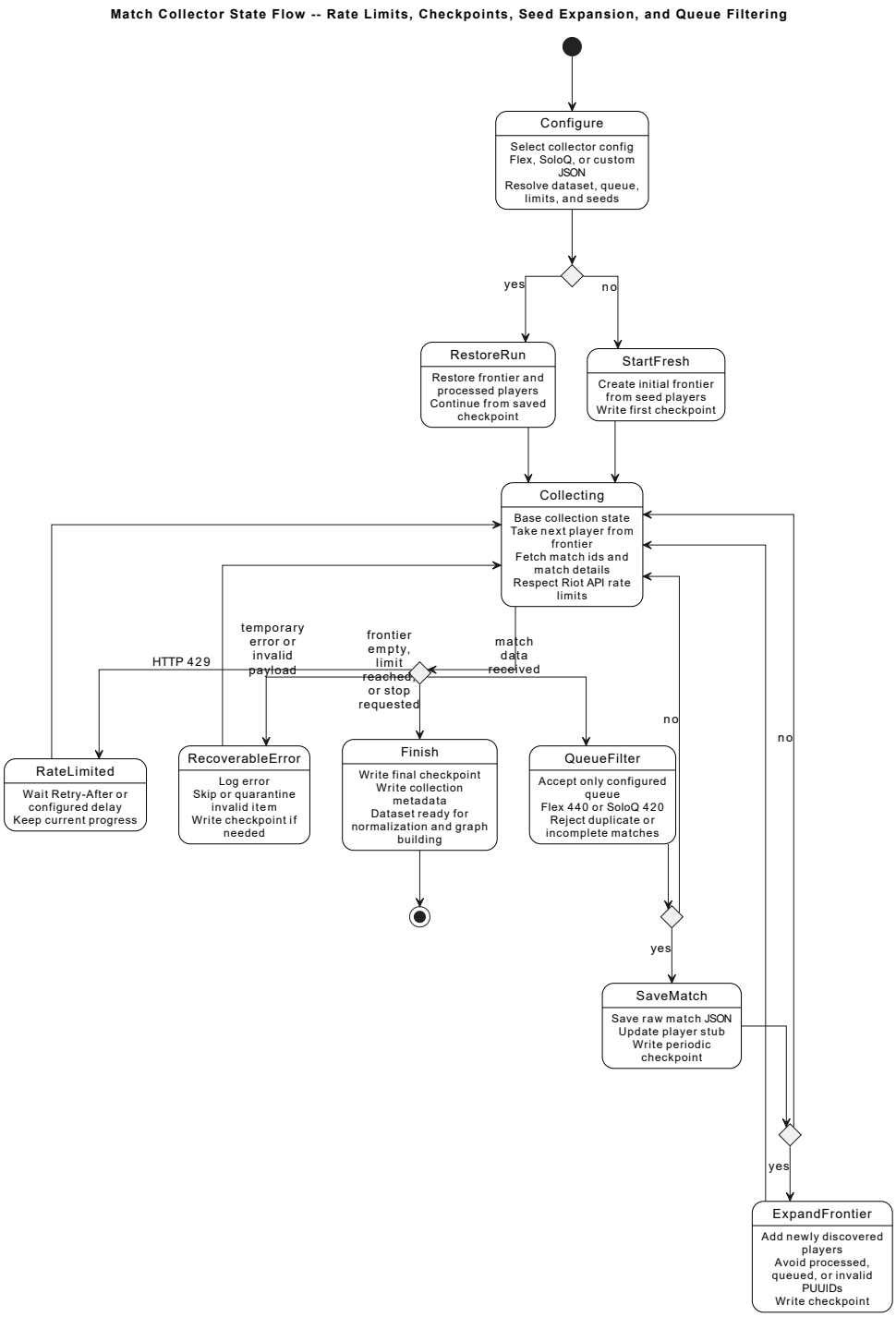


Figure 3.1: Flow diagram of the collector's parameterisable state machine

3.3 Collection Focus and Queue Selection

The project uses two research datasets: *flexset* consists of Ranked Flex Queue matches, and *soloq* consists of Ranked Solo/Duo Queue matches. The queue type choice is not arbitrary. Ranked Flex allows premade teams to participate, so recurring ally relationships can appear naturally. Ranked Solo/Duo intentionally excludes 5-player premades, making it a controlled comparison baseline.

3.4 Rate Limits and Reliability

The Riot API's constraints also shape the architecture. The collection layer gained explicit rate-limit handling, wait logic, and subtask splitting. The collector applies its own per-second wait, waits thirty seconds on HTTP 429 before retrying, and skips already-saved matches based on file existence.

Table 3.1 summarises the collector's five main reliability mechanisms and their roles.

Table 3.1: Main reliability mechanisms of the collector

Mechanism	**Function**
Per-second delay	Controls short-term API load
Two-minute counter	Aligns with Riot key usage limits
429 retry	More robust operation on transient limit hits
Existing match file skip	Avoids duplicate saves and re-downloads
Event-based stdout log	Supports status tracking on both backend and frontend

These mechanisms are not mere technical details: collecting the full *flexset* (approximately 4980 matches) with a development API key took days, with interruptions. Without file-based skipping and restartability, the corpus could not have been assembled without a continuously running server. The collector's design decisions are therefore a direct response to the API constraints: iterated, stateful batch runs rather than a single-pass download. After each overnight run, collection resumed exactly where it left off.

3.5 Comparison of flexset and soloq Collection Strategies

The *flexset* dataset contains exclusively queue ID 440 (Ranked Flex) matches from EUNE Master+ ladder seed players. The collector's premade-probe mechanism filters out candidates that lack sufficient recurring teammate patterns.

The *soloq* dataset consists of queue ID 420 (Ranked Solo/Duo) matches from an EUNE Master+ Solo/Duo ladder seed population. The role of *soloq* is to serve as a control baseline: a dataset in which recurring premade connections are systematically not favoured.

Table 3.2 compares the main features of the two dataset collection strategies.

Table 3.2: Comparison of the two dataset collection strategies

Dimension	**flexset**	**soloq**
Queue ID	440 (Flex)	420 (Solo/Duo)
Premade-probe	Yes, based on repetitions	No
Seed expansion	Via teammates	Via list retrieval
Collection goal	Social structure	Performance baseline
Corpus size	4980 matches	2000 matches
Role in project	Main research dataset	Control dataset

When interpreting Riot API data it is important to keep in mind that the network is *not a representation of the entire League of Legends universe*, but a bounded sub-world created by a specific collection strategy. Detailed data quality and bias analysis can be found in chapter 4 of the extended version [7].

4. System Architecture and Evolution

4.1 The Early Version

The original system was much closer to a data-driven player viewer than the current research platform. The early scoring logic (*add_new_players.js*) used simple formulas. The early *build_graph.py* was Python-centred: it built a relationship graph from matches and produced HTML and PyVis exports. It was along this path that cluster profiles eventually became the basis for the Tribal NeuroSim simulation input.

4.2 The Current System

The system is built on several well-separated layers. Its components are: Python (data collection, graph construction, legacy exports), SQLite (central persistence), Node.js/Express (API shell and orchestration), Rust (pathfinding, graph analytics, global artifact export, simulation logic), React/Vite (frontend and visualisation), C# ASP.NET Core (standalone database browser), and C# MonoGame (the Tribal NeuroSim visualisation layer, which renders Rust payloads).

The component diagram can be found in chapter 5 of the extended version [7]; in the lite version the general description is provided in the text.

4.3 Technology Stack Rationale

Every technology choice had alternatives; the following lists the main selection criteria for each layer and the competitors considered.

Python (data collection, scripting layer). Rich ecosystem (Riot API client, NetworkX, pandas), short iteration cycle, easily modifiable collection logic. Alternative: Go or Ruby scripts would have offered a faster start, but the scientific ecosystem (NetworkX, scipy) is tied to Python. Downside: weaker performance on large graph operations, which the Rust layer compensates for.

SQLite (persistence). Serverless, self-contained file, easy version control: both Python and Rust support it natively. Alternative: PostgreSQL would have provided stronger concurrent writes and more complex queries, but the project is a single-user, locally runnable, demonstrable system. The SQLite trade-off is more favourable at this scope [4]. Downside: no concurrent

writes, and schema changes require migration.

Node.js / Express (API shell). The frontend (React) and the backend share the same ecosystem; Express easily proxies the Rust binary and Python scripts. Alternative: FastAPI (Python) would have been more natural alongside the Python layer, but since the Python layer was downgraded to data collection and simple aggregations only, Node provided better async I/O for Riot API relay calls and simpler Vite integration. Downside: CPU-intensive computations do not run in Node.

Rust (graph analytics, pathfinding). Zero runtime overhead, safe memory management, Rayon parallelism, exact determinism [35, 36]. Alternative: C++ would yield equal performance, but the Rust borrow checker prevents invalid-iterator errors that are common in graph-traversal code. Go would offer an easier concurrency model but weaker cache performance. Downside: steeper learning curve, a demanding compilation experience, and longer build times.

React / Vite (frontend). Component-based UI, large ecosystem (MUI, Three.js), fast hot reload. Alternative: Vue or Svelte would produce a smaller bundle, but the MUI and Three.js React integrations are more mature. Downside: overkill for a single-page demo; the project requires multiple visualisation pages, which justifies the framework.

Table 4.1 summarises the rejected architectural approaches and the reasons for rejecting them.

Table 4.1: Rejected approaches and rationale

Alternative	**Brief advantage**	**Why it was not chosen**
Everything in Python	Faster initial iteration	Weaker performance and memory control for runtime graph algorithms and larger exports
Everything in Rust	Strong typing and a unified engine	Collection and rapid experimentation would have become significantly harder and slower
Frontend-centred computation	Simpler demo architecture	Would have pushed compute-intensive graph work to the browser, weakening reproducibility
Server-side SQL database	Stronger multi-user model	Given the project's local, demonstrable, research-oriented nature, SQLite remained the better trade-off

4.4 Multi-Dataset Support

The implementation manages multiple independent datasets. Based on the *backend/data/-datasets.json* registry, each dataset has its own raw database, refined database, match JSON directory, and cache directory.

Dataset selection does not stop at the Node layer. The backend derives environment variables from the selected dataset for the Rust runtime (*DATASET_ID*, *GRAPH_DB_PATH*, *PATHFINDER_MATCH_DIR*, *PATHFINDER_BIRDSEYE_CACHE_DIR*).

Figure 4.1 shows the complete lifecycle of a dataset, from Riot API data sources to analytical outputs.

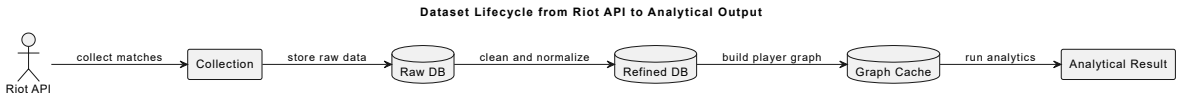


Figure 4.1: Lifecycle diagram of a dataset from the Riot API to analytical output

4.5 Current Data State

The data state was recorded on 28 April 2026 by directly inspecting the repository.

Table 4.2 records the active data state as of 28 April 2026; no dataset expansion took place after that date.

Table 4.2: Current data state of the two datasets

Metric	**flexset**	**soloq**	
Match JSON files	4 981	2 001	
Refined players	15 421	6 768	
Clusters	599	140	
Cluster members	5 741	1 538	

The soloq dataset clusters typically represent *duo pairs*: since Solo/Duo Queue allows premades of at most two players, the vast majority of clusters that emerge from recurring connections contain only two or three players.

4.6 Persistence and the Relational Model

Clusters previously existed only in JSON export form; database integration made them first-class database objects. The schema is built around three central tables [4]: *players*, *clusters*, and *cluster_members*.

Figure 4.2 shows the entity-relationship diagram of the main SQLite database, including the *players*, *clusters*, and *cluster_members* tables.

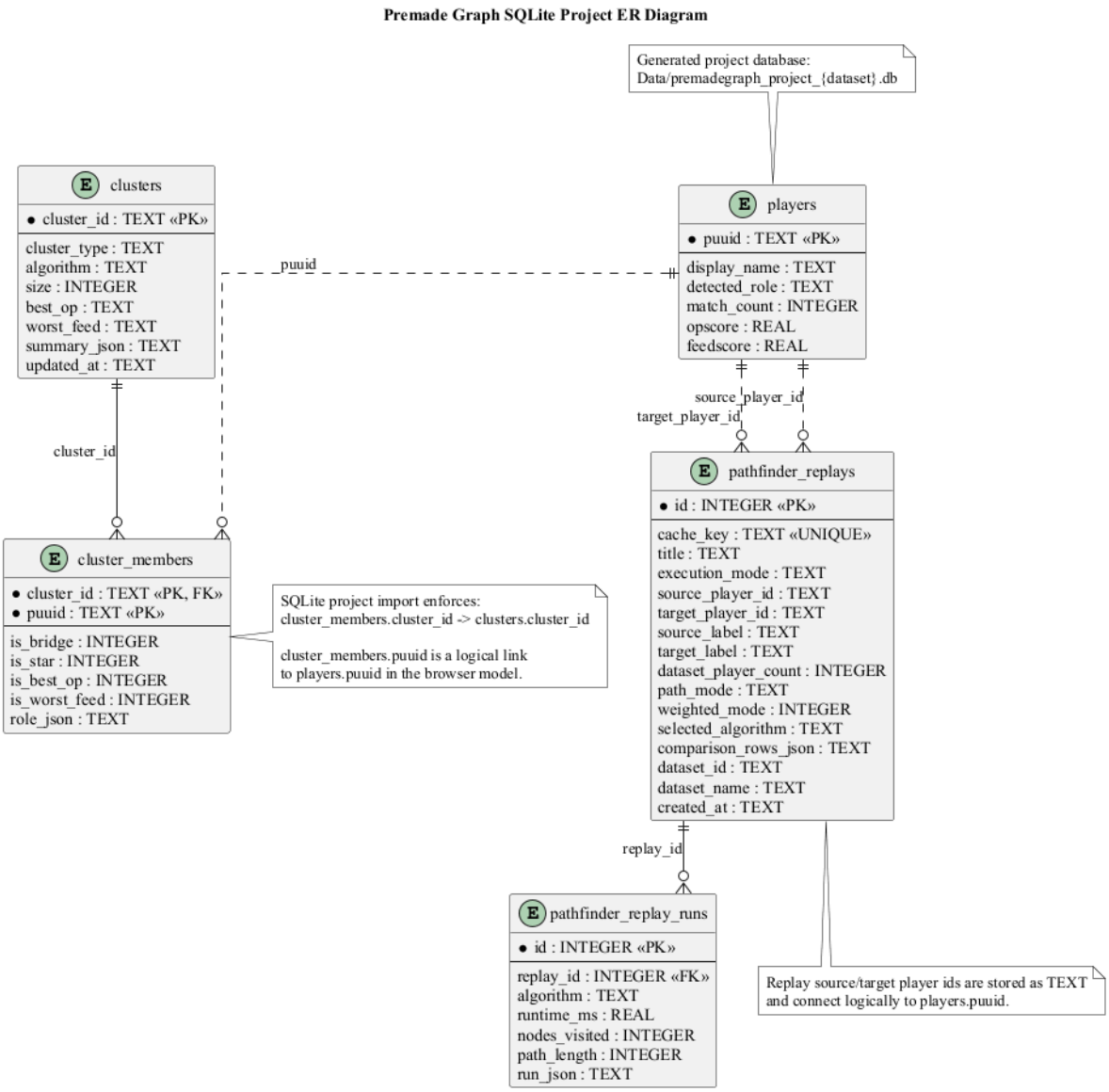


Figure 4.2: Entity-relationship diagram of the main SQLite database

Figure 4.3 shows the relational model, highlighting the relationships between the three tables.

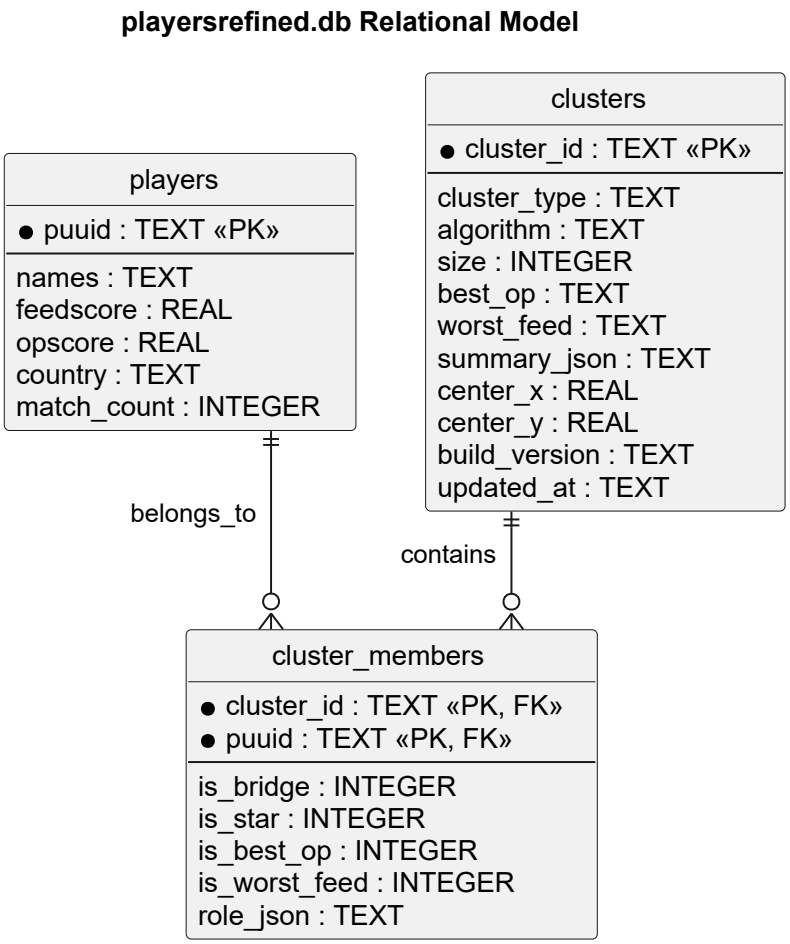


Figure 4.3: Relational model for cluster and player persistence (*players*, *clusters*, *cluster_members*)

Two cluster families coexist in the system. *python_population* type clusters are the output of the Python graph-building pipeline: NetworkX-based modularity maximisation with a community-detection perspective. *rust_pathfinding* type clusters come from the Rust engine's runtime clustering: tightly connected segments based on ally connections, capped at 20 members. The aggregated performance metrics of the cluster profiles also form the genome sequence input for the Tribal NeuroSim simulation.

5. Comparing the Two Datasets: flexset and soloq

The analysis is built on datasets collected from two different queue types: the different matchmaking mechanics of Flex Queue and Solo/Duo Queue produce inherently different graph structures. When the same pipeline is run on both datasets, the difference is one of queue mechanics rather than model parameters.

5.1 Structural Differences Between the Two Datasets

Flex Queue (queue ID 440) allows team formation: players can form premade teams of 2 to 5. This means that some player pairs appearing repeatedly on the ally side represent *intentional repetition*. The ally/enemy ratio (73.17% ally in *flexset*) is partly a consequence of this natural mechanism, since collecting premade connections was the goal in both cases.

Solo/Duo Queue (queue ID 420) allows premades of at most two players. The ally/enemy ratio (65.24% ally in *soloq*) is lower, consistent with more randomly assigned matches producing fewer dominant ally connections. A third or even fourth player may appear in certain clusters, since a seeded player does not necessarily play exclusively with one partner throughout a climb.

Table 5.1 contains the basic size metrics of the two datasets.

Table 5.1: Basic sizes of the two datasets

Metric	**flexset**	**soloq**	
Match JSON files	4981	2001	
Raw players	15 147	2822	
Refined players	15 421	6768	
Clusters (DB)	599	140	
Cluster members (DB)	5741	1538	
Visible nodes	5741	1538	
Visible edges	18 548	4111	
Average edge weight	3.405	2.364	
Ally/enemy ratio	73.17% / 26.83%	65.24% / 34.76%	
Visual clusters	1531	870	
Average cluster size	3.750	1.768	
Largest cluster size	20	20	

The average cluster size in *flexset* (3.750) is more than twice that of *soloq* (1.768): the former is

dominated by groups of 3–4, the latter by groups of 1–2. The queue mechanics directly explain this: Flex Queue allows premades of up to 5, while Solo/Duo Queue mainly involves pairs.

5.2 Summary

Table 5.2 summarises the comparison of the two datasets along the main dimensions.

Table 5.2: Summary comparison of the two datasets

Dimension	**flexset**	**soloq**
Queue mechanics	Flex, premade-focused	Solo/Duo, individual-centred
Average edge weight	3.405 (denser)	2.364 (sparser)
Ally ratio	73.17%	65.24%
Average cluster size	3.750	1.768
Triangles/node†	4.95	1.03
Core-periphery structure‡	Bridge-rich associative core with many peripheral ally islands	Smaller, more randomly distributed local groups
Social-path opscore assortativity	0.2346	0.1750
Battle-path opscore assortativity	0.1461	0.1625
Feedscore battle-path assortativity	-0.0108	-0.0366
Research role	Associative graph and performance relationship	Performance baseline, control

† *Triangle structures* (triads): if player A knows B and C, and B also knows C, this forms a closed triangle in the network. More such closed triangles indicate stronger group cohesion. ‡ *Core-periphery*: the inner core of the graph is densely connected, while peripheral nodes have only a few connections.

flexset is the more natural setting for the associative player graph and recurring ally connections; *soloq* serves as the performance baseline and control dataset. The *battle-path opscore* value is higher in *soloq* (0.1625) than in *flexset* (0.1461). In *social-path* mode *flexset* is stronger (0.2346 vs. 0.1750), reflecting the premade-ally mechanism. Further discussion can be found in chapter 6 of the extended thesis [7].

6. The opscore Performance Metric

The foundation of graph analytics is a meaningful, role-aware performance score. The raw data from League of Legends matches (kills, deaths, gold, damage) carries insufficient semantics on its own: a support's second death does not have the same significance as an ADC's second death. The data collection and PUUID normalisation mechanisms are described in chapter 4 of the extended version; this chapter focuses on the internal model and formulas of the opscore metric.

6.1 Metric Design

6.1.1 The Role of a Custom Performance Metric

External services such as OP.GG offer player analytics features and easily interpretable performance ratings developed over many years. In this project, however, player evaluation is performed directly from match data collected via the Riot API, using a custom *opscore* metric. The OP.GG methodology provided significant inspiration for the design of this metric [3].

From the project's perspective, external performance ratings would have operated as black boxes, since their exact computation, weighting, and normalisation are not publicly accessible. Consequently, scores published by OP.GG cannot be reproduced from the raw match data collected via the Riot API. The custom metric, by contrast, allows performance scores to be reproduced, the role-aware weighting and normalisation to be verified, and player values to be traced directly back to the match data used.

6.1.2 Limitations of the Earlier Model

The model was developed through gradual methodological refinement. The starting point was an eight-artifact system that applied role multipliers and then normalised the resulting raw sums to the $[0, 10]$ range relative to the percentile distribution in the database.

Two fundamental weaknesses surfaced early. Database-relative normalisation: if the database is dominated by very strong players, even a median player scores around the middle of the scale. Death-based approximation: the earlier system could not distinguish between the roles of tank, support, and feeder.

6.2 The Local, Role-Aware opscore Model

The current *opscore* model operates on the following design principles: local, database-independent evaluation; five artifacts, ten subcategories; logarithmic boosting for elite performance; reliability handling for short matches.

6.3 The Five Artifacts and Their Subcategories

Figure 6.1 shows the complete structure of the five artifacts and ten subcategories, including the role-aware weighting.

The model is built from five artifacts. **Combat Impact** measures combat conversion and damage pressure; **Risk Discipline** captures the cost of deaths and survival quality. **Resource Tempo** tracks economic efficiency and early-game rhythm. The remaining two dimensions cover team-level impact: **Map and Objective Control** for map and objective control, and **Team Enablement** for value created for teammates.

6.4 Mathematical Formalism

6.4.1 The Role Ratio and the Base Curve

The starting point for scoring each subcategory is the ratio of the observed value to the role expectation:

$$r = \frac{x}{E_{\text{role}}} \tag{6.1}$$

The base curve is a two-parameter, S-shaped response function:

$$m = \left(\frac{1-t}{t}\right)^{1/h} \tag{6.2}$$

$$B(r) = \frac{r^h}{r^h + m^h} \tag{6.3}$$

where $t = 0{,}70$ and $h = 2$. The curve is constructed so that $B(1) = t$.

The two parameters of the S-curve encode a specific intuition: a player who performs exactly at the expected level ($r = 1$) always receives $B(1) = 0{,}70$. This means the on-expectation player sits at 70% of the scale — not below threshold, but not excellent either. The logarithmic boost term ($\lambda \cdot \ln r$) elevates those who far exceed the role expectation: a flatter curve could not distinguish a player at 120% from one at 300%.

For positive metrics:

$$S_+(r) = \min(c,\ B(r) + \lambda \cdot \ln(\max(1,\ r))) \tag{6.4}$$

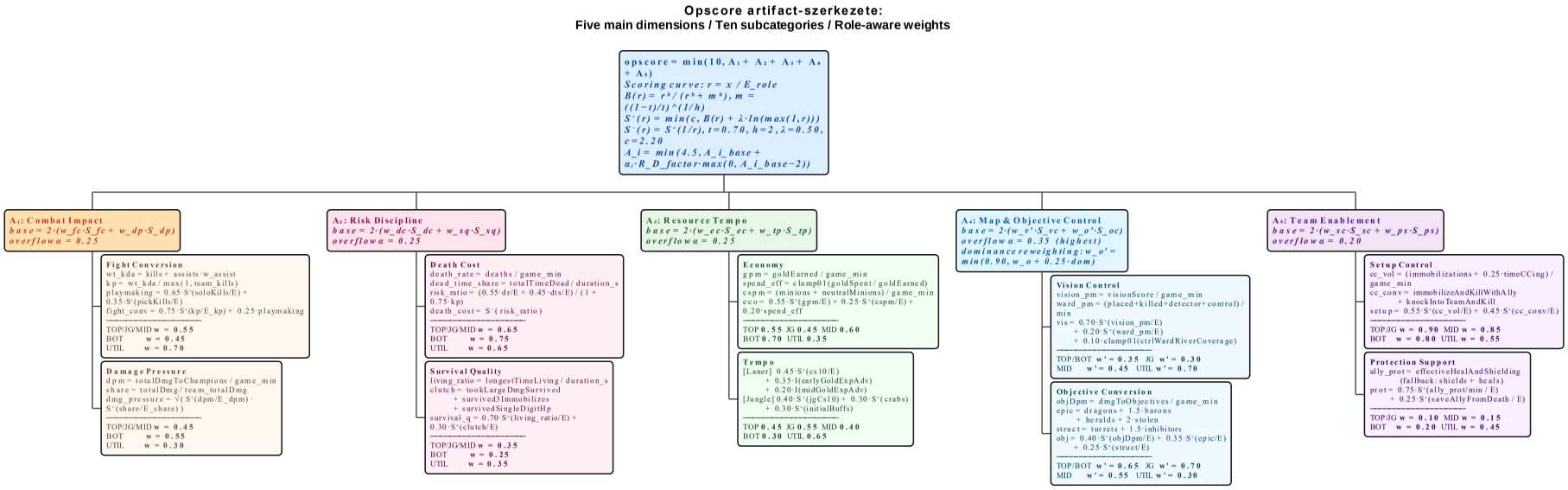


Figure 6.1: The opscore artifact structure: five main dimensions, ten subcategories, role-aware weights

where $\lambda = 0{,}50$ and $c = 2{,}20$. For negative metrics: $S_-(r) = S_+\left(\frac{1}{r}\right)$.

6.4.2 Duration Reliability

$$R_D = \mathsf{clamp}_{[0,1]}\left(\frac{D - 12}{8}\right) \tag{6.5}$$

where D is match length in minutes. For per-minute metrics sensitive to variance:

$$\tilde{S} = R_D \cdot S + (1 - R_D) \cdot 0{,}70 \tag{6.6}$$

6.4.3 Artifact Aggregation and Boosting

$$A_i^{\mathsf{base}} = 2\,(w_1 \cdot S_1 + w_2 \cdot S_2) \tag{6.7}$$

$$\gamma = 0{,}5 + 0{,}5 \cdot R_D \tag{6.8}$$

$$A_i = \mathsf{min}\big(4{,}5,\ A_i^{\mathsf{base}} + \alpha_i \cdot \gamma \cdot \mathsf{max}\big(0,\ A_i^{\mathsf{base}} - 2\big)\big) \tag{6.9}$$

Table 6.1 contains the boost parameters (α_i) for the five artifacts.

Table 6.1: Artifact-specific boost parameters (α_i)

Artifact	$\boldsymbol{\alpha_i}$
Combat Impact	$0{,}25$
Risk Discipline	$0{,}25$
Resource Tempo	$0{,}25$
Map and Objective Control	$0{,}35$
Team Enablement	$0{,}20$

6.4.4 The Final opscore

$$\mathsf{opscore} = \mathsf{min}\left(10,\ \sum_{i=1}^{5} A_i\right) \tag{6.10}$$

6.4.5 Exceptional Performances and Edge Cases

The model is best illustrated through two characteristic edge cases.

Elite carry match. During development, a standout Quinn Top match was examined (exceptional kill count and high assists), where the earlier model could be constrained by hard artifact

ceilings and caps, resulting in an average, unremarkable score. In the current model, the logarithmic boost of the *Combat Impact* and *Map and Objective Control* artifacts allows these values to rise above the nominal 2-point ceiling; the aggregate *opscore* reached the 10-point cap. The cost of deaths does not disappear: the *Risk Discipline* artifact penalises them moderately, but the overall performance of an elite carry remains recognised.

Split-pusher and structure carry. A player on *Yorick* who deliberately avoids teamfights and wins by destroying towers typically has a low *Team Enablement* score and an unremarkable KDA, since the strategy is built around split-pushing. The dynamic dominance re-weighting of the *Map and Objective Control* artifact is calibrated precisely for this case: when Objective Conversion dramatically exceeds the Team Enablement value, the artifact shifts weight toward objective acquisition, and the low team-enablement number does not unduly distort the final result when the win condition has genuinely been delivered. Detailed edge cases and sample-size limitations can be found in the extended version [7].

6.5 The Role of opscore in the System

The *opscore* and *feedscore* values appear in two places in the system. Assortativity and centrality calculations use them as node attributes: the question is whether connected players carry similar performance profiles. In the Tribal NeuroSim simulation, cluster-level aggregated scores (*A_combat*, *A_resource*, *A_map_objective*, *A_risk*, *A_team*) are computed directly from the five artifacts and form the initial genome sequence of each tribe — the performance profile derived from match data drives the behavioural tendencies exhibited in the simulation.

7. Player Network Construction, Search, and Measurements

7.1 Graph Construction

The primary graph state is produced by the Rust layer. The Python side retains its role in data collection, rapid prototyping, and compatibility graph generation. The *flexset*/*soloq* comparison, assortativity analysis, and graph visualisation all rely on the Rust-based graph model.

7.1.1 Edge Aggregation

The Rust graph model stores ally-side and enemy-side evidence separately for each player pair:

$$w_{ally}(u,v) = \sum_{m \in M} \mathbf{1}[\exists i \in \{1,2\} : u \in T_{m,i} \wedge v \in T_{m,i}] \tag{7.1}$$

$$w_{enemy}(u,v) = \sum_{m \in M} \mathbf{1}[\exists i \neq j : u \in T_{m,i} \wedge v \in T_{m,j}] \tag{7.2}$$

The dominant relation is *ally* if *allyWeight* $\geq$ *enemyWeight*, otherwise *enemy*. The graph results should be interpreted primarily as an associative player graph, not as a literal friendship or hostility network.

7.1.2 The Rust Graph State

The Rust layer builds the graph directly from match JSON files and the player database. *GraphState* stores adjacency lists alongside the *pair_relations* structure, fast-access player profiles, cluster membership, landmark distances, minimum costs, and snapshot metadata for the active dataset.

7.1.3 Filtering and the Research Graph

The visibility threshold for the research graph is *totalMatches* ≥ 2: a single shared match is not sufficient for a connection to appear as a stable edge. The primary clustering direction uses the Rust-side deterministic *bounded-ally-groups* logic. Visual cluster sizes are intentionally capped

at roughly twenty players.

7.1.4 The Graph Artifact Layer

The Rust graph-export pipeline writes pre-computed, dataset-bound cache artifacts: *manifest.json* records the dataset and the graph-builder version; binary files (*node_positions.f32*, *edge_pairs.u32*) are the input files for the Three.js/WebGL layer; *cluster_meta.json* contains cluster identifiers. This artifact layer is the authoritative source of truth for research visualisation.

7.2 Pathfinding and Visualisation

7.2.1 Search Algorithms

The current Rust runtime supports four algorithms: BFS, Dijkstra, bidirectional search, and A*. The algorithms can run in two edge interpretations: *social-path* traverses ally connections only, *battle-path* traverses all recurring connections.

Edge cost model: in unweighted mode every edge has a fixed cost of 1000. In weighted mode:

$$c(u, v) = \left\lceil \frac{1000}{w(u, v)} \right\rceil \tag{7.3}$$

where $w(u, v)$ is the connection strength corresponding to the active path mode.

Table 7.1 compares the characteristics of the four search algorithms in the Pathfinder Lab.

Table 7.1: Algorithm comparison in the Pathfinder Lab

Algorithm	**Natural fit**	**Notes in this project**
BFS	Unweighted shortest path	Good baseline, easily explained
Dijkstra	Weighted paths	Reference algorithm for cost-based modes
Bidirectional	Larger graph, less directed search	Practical speedup in some cases
A*	Well-formed lower bound	Remains exact in this project, not just a visual heuristic

7.2.2 Exact A* and the ALT Heuristic

The ALT (A* + Landmarks + Triangle inequality) heuristic [19] is based on the principle that for any landmark L:

$$d(u, t) \geq |d(L, t) - d(L, u)| \tag{7.4}$$

for every landmark L. Landmark distances are pre-computed during *GraphState* construction for every *ModeKey* combination and reused across all search requests. A* produces correct shortest paths; unit tests confirm that A* results match Dijkstra results in all relevant cases.

7.2.3 Pathfinder Lab

The Pathfinder Lab is an interactive testing environment for the four search algorithms: results can be replayed step by step, and the two path modes (social-path, battle-path) can be compared directly.

Figure 7.1 shows the Pathfinder Lab replay panel, including the path summary, step controls, and visited-node statistics.

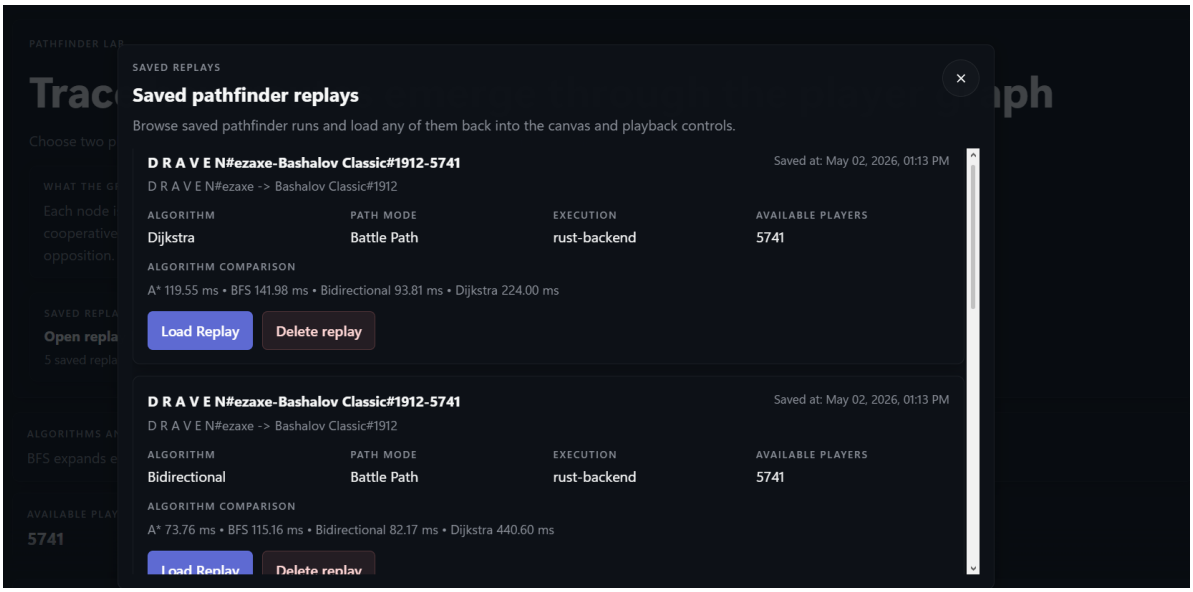


Figure 7.1: The Pathfinder Lab replay panel: the path summary, step controls, and visited-node statistics

Table 7.2: BFS vs. weighted search: same player pair, different notions of proximity

Algorithm	**Metric**	**Result**	**Interpretation**
BFS	Minimum hops	3	Shortest social chain
Dijkstra	Min. weighted cost	8	Strongest ally path
A*	Min. weighted cost	8	Matches Dijkstra
Bidirectional	Minimum hops	3	Matches BFS

The 3-hop path found by BFS structurally exists but passes through weak or single-match connections. Dijkstra in weighted mode avoids these and finds an 8-hop path on which every edge is a recurring ally relationship.

Table 7.3 compares the performance characteristics and main strengths of the four algorithms.

Table 7.3: Comparison of the four algorithms along the key dimensions

Algorithm	**Optimal?**	**Memory**	**Key strength in this project**
BFS	Yes (unweighted)	$O(n)$	Simple baseline, easily explainable replay
Dijkstra	Yes (weighted)	$O(n+m)$	Reference algorithm, lexicographic tie-breaking
Bidirectional	Yes (unweighted)	$O(n)$	Potential speedup on clustered graphs
A*	Yes (admissible heuristic)	$O(n)$	Goal-directed traversal, exact result

7.2.4 The 3D Global Visualisation

The *bird's-eye sphere* provides a visual overview of the entire network. The Rust layer pre-exports all major artifacts, and the frontend only renders.

Table 7.4 presents the artifacts of the bird's-eye export and their roles.

Table 7.4: Main artifacts in the bird's-eye export

File	**Role**
manifest.json	Metadata, counts, scaling and generation info
node_meta.json	Node-level descriptive metadata, frontend mappings
node_positions.f32	Float coordinate array for fast rendering
node_metrics.u32	Compressed numeric node metrics
edge_pairs.u32	Edge endpoint pair array
edge_props.u32	Encoded per-edge properties

Figure 7.2 shows the complete data flow of the graph artifact package, from the Rust exporter to the Three.js renderer.

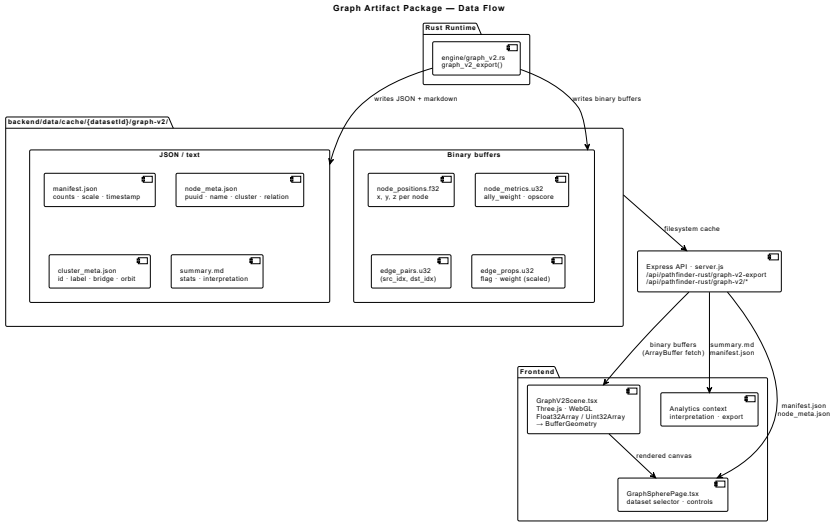


Figure 7.2: Data flow of the graph artifact package: the Rust exporter writes binary and JSON layers, Express serves them, and the frontend consumes them in a WebGL renderer

Figure 7.3 shows the structure of the binary and JSON files in the artifact package.

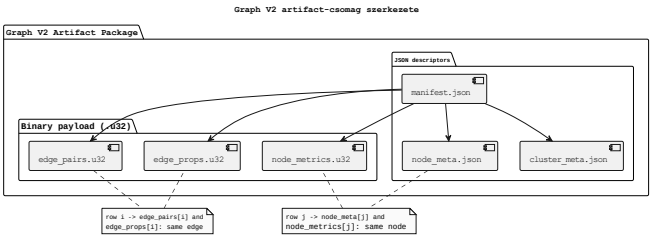


Figure 7.3: Structure of the binary and JSON artifact files in the graph-export pipeline

Figure 7.4 shows the 3D bird's-eye view of the entire *flexset* network, rendered from pre-generated Rust artifacts.

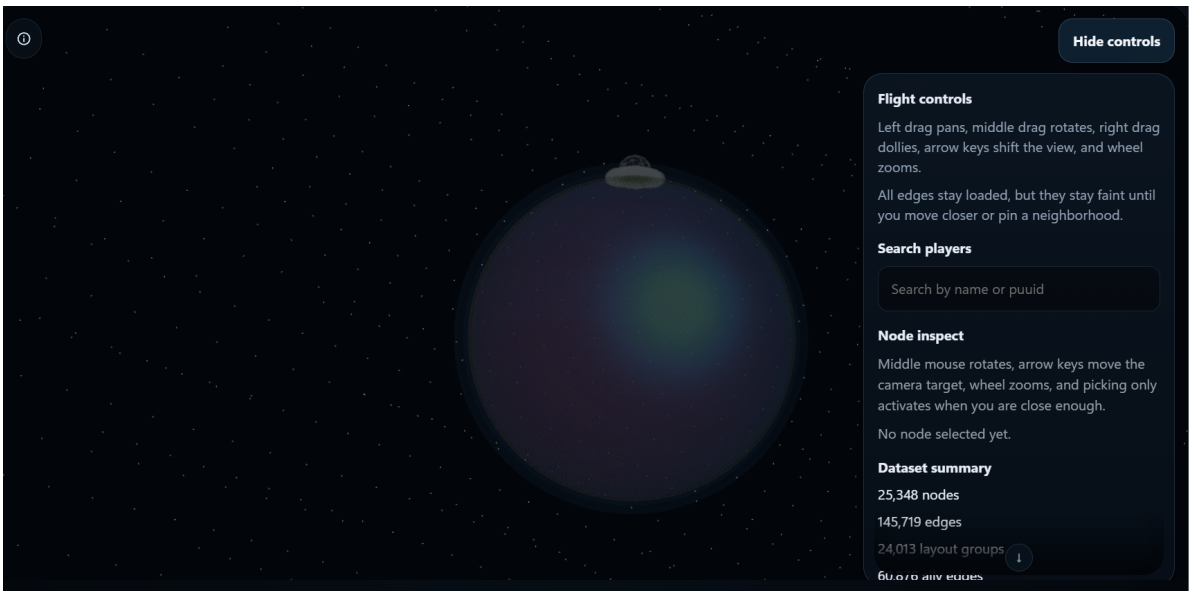


Figure 7.4: 3D bird's-eye view of the full network, built on pre-generated Rust artifacts

Figure 7.5 shows the denser cluster regions of the *flexset* graph and their spatial separation in the global view.

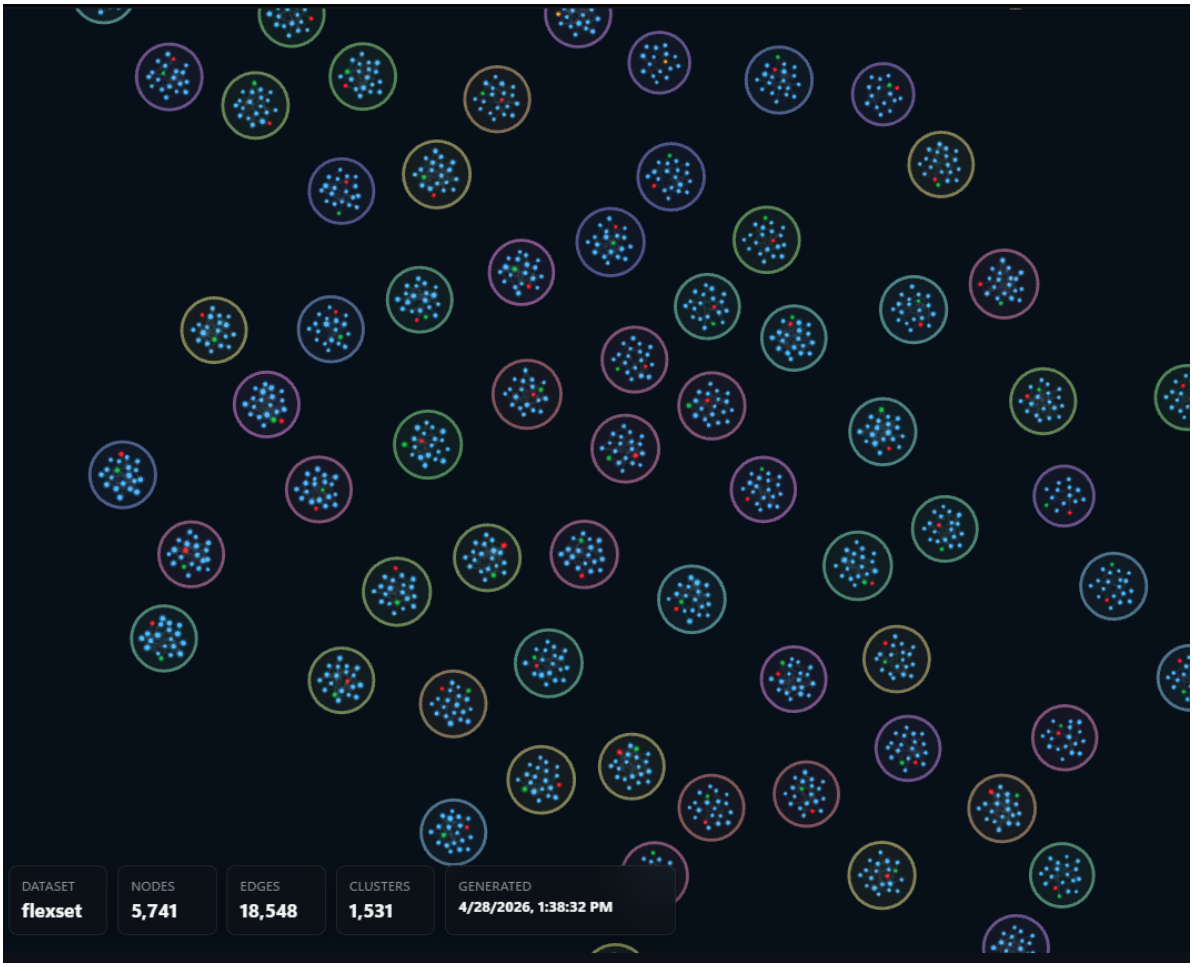


Figure 7.5: Denser cluster regions and spatial separation in the global view

7.3 Assortativity

7.3.1 Computational Definition

The implementation computes numeric assortativity as Pearson correlation:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \tag{7.5}$$

Undirected edges are included by the implementation in both endpoint orientations. Edge weighting by graph mode:

$$w(u,v) = \begin{cases} \textit{ally_weight} & \text{if } \textit{social-path} \\ \textit{total_matches} & \text{if } \textit{battle-path} \end{cases} \tag{7.6}$$

7.3.2 Assortativity Results: flexset

Table 7.5 contains the *flexset* assortativity results for the two graph modes.

Table 7.5: Assortativity: *flexset*

Graph mode	Metric	Correlation	Sample size
social-path	*opscore*	0.2346	61 938
social-path	*feedscore*	0.1816	61 938
battle-path	*opscore*	0.1461	179 492
battle-path	*feedscore*	-0.0108	179 492

Figure 7.6 visualises the *flexset* social-path assortativity analysis for *opscore* and *feedscore*.

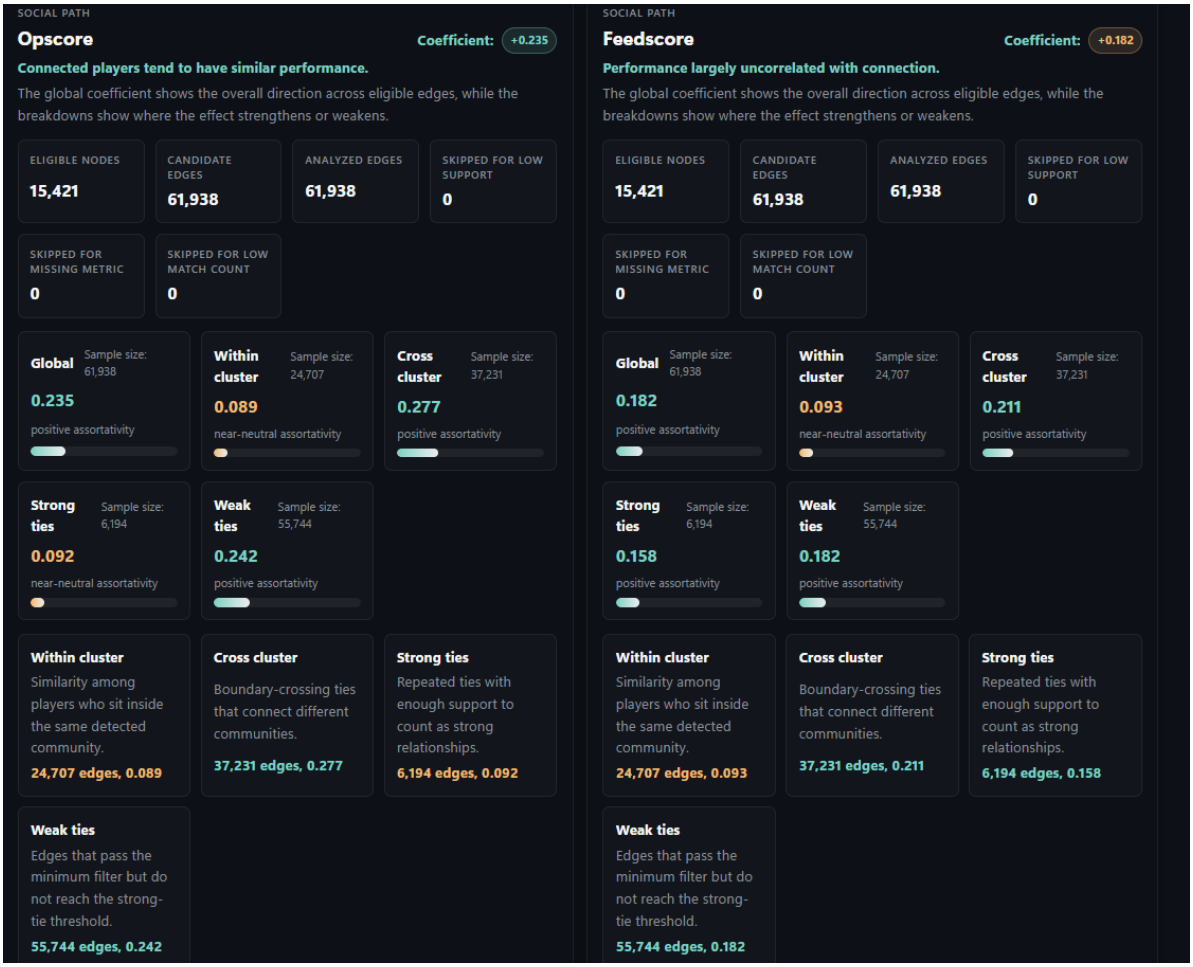


Figure 7.6: *flexset* social-path assortativity by *opscore* and *feedscore*. 15 421 eligible nodes and 61 938 analysed edges; global coefficient 0.235 for *opscore* and 0.182 for *feedscore*.

7.3.3 Assortativity Results: soloq

Table 7.6 contains the *soloq* assortativity results, comparing the two graph modes.

Table 7.6: Assortativity: *soloq*

Graph mode	Metric	Correlation	Sample size
social-path	*opscore*	0.1750	37 976
social-path	*feedscore*	0.1012	37 976
battle-path	*opscore*	0.1625	84 393
battle-path	*feedscore*	−0.0366	84 393

Figure 7.7 shows the *soloq* social-path assortativity results.

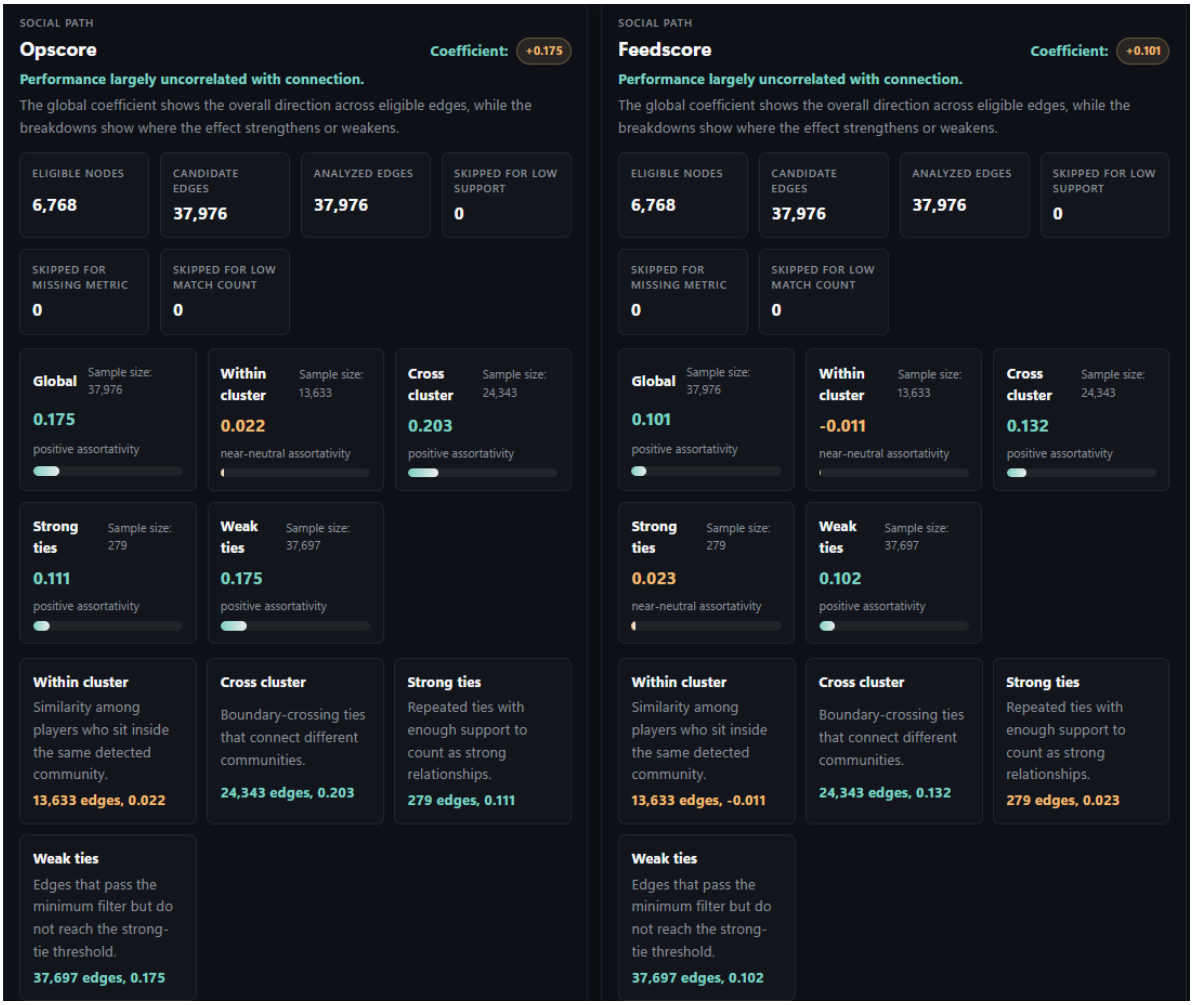


Figure 7.7: *soloq* social-path assortativity by *opscore* and *feedscore*. 6 768 eligible nodes and 37 976 analysed edges; global coefficient 0.175 for *opscore* and 0.101 for *feedscore*.

7.3.4 Interpretation

Positive assortativity along *opscore* in *social-path* mode appears in both datasets, indicating that in the graph built on ally connections, the performance profiles of connected players are not randomly mixed. The behaviour of *feedscore* is much more sensitive to graph mode: a moderate positive signal appears on both datasets in social-path mode, but it falls close to zero in battle-path mode.

opscore remains positive across all tested modes while *feedscore* does not; this is a strength of the composite metric over simpler death-centred formulas.

7.4 Betweenness Centrality

7.4.1 The Brandes Algorithm

The Brandes algorithm [20] reduces the traditional $O(n^3)$ betweenness computation to $O(nm)$. For each source node: run Dijkstra/BFS, then in the back-propagation step:

$$\delta[s][v] \mathrel{+}= \frac{\sigma[s][v]}{\sigma[s][w]} \cdot (1 + \delta[s][w]) \tag{7.7}$$

In weighted mode, edge cost is:

$$c(u, v) = \left\lceil \frac{10^6}{\mathsf{max}(1, w(u, v))} \right\rceil \tag{7.8}$$

7.4.2 Parallel Execution

The Rust *rayon* library enables automatic work-stealing parallelism. The centrality module splits the source-node set into $k = \mathsf{min}(n, 4 \cdot \text{thread count})$ equal-sized chunks. Deterministic ordering of source nodes and fixed chunk boundaries guarantee that identical input always produces identical output.

7.4.3 Result

Figure 7.8 shows the Brandes algorithm result screen, including the comparison of parallel and serial runtimes.

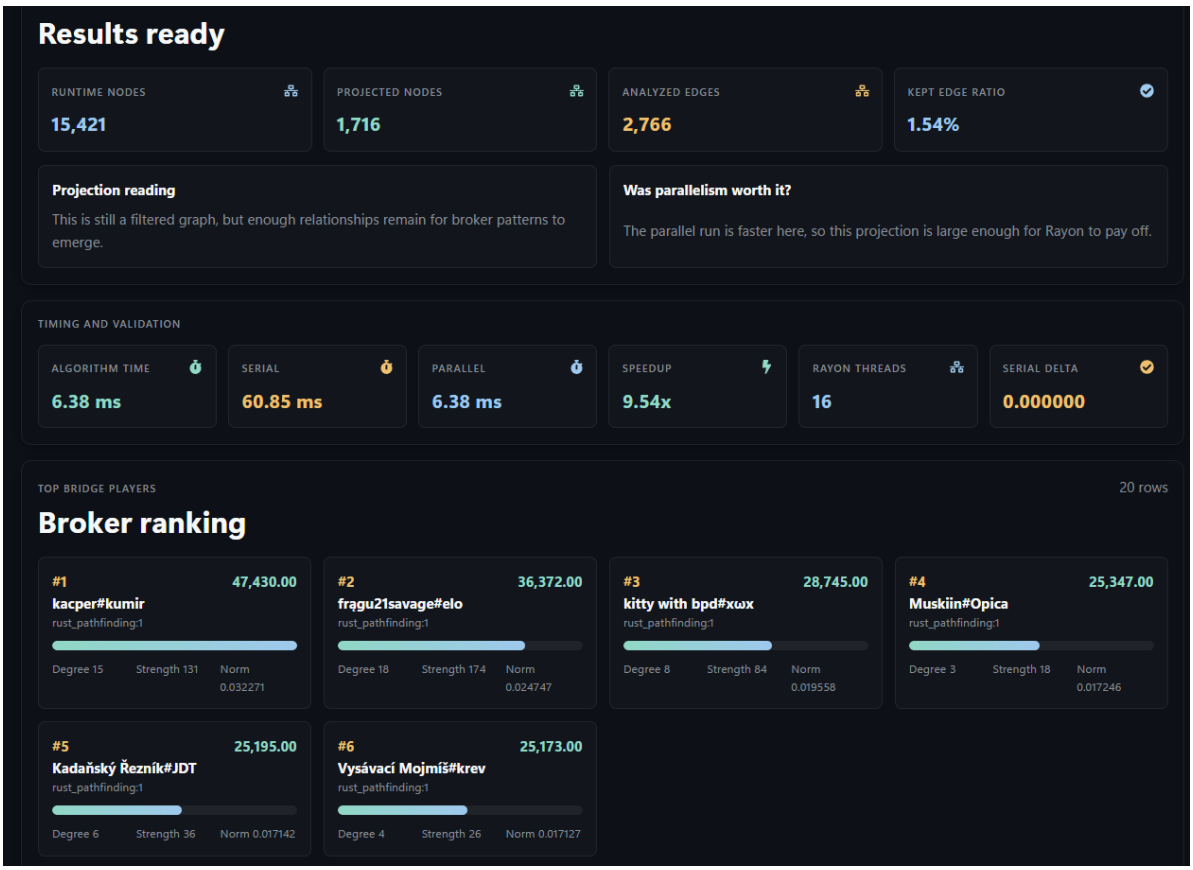


Figure 7.8: Brandes betweenness centrality result screen. 15 421 runtime nodes; 1 716 projected nodes and 2 766 analysed edges. Parallel run: 6.38 ms; serial baseline: 60.85 ms; speedup: 9.54×; 16 Rayon threads; serial–parallel delta: 0.000000.

Figure 7.9 shows the players with prominent brokerage roles according to centrality, highlighted in yellow in the graph.

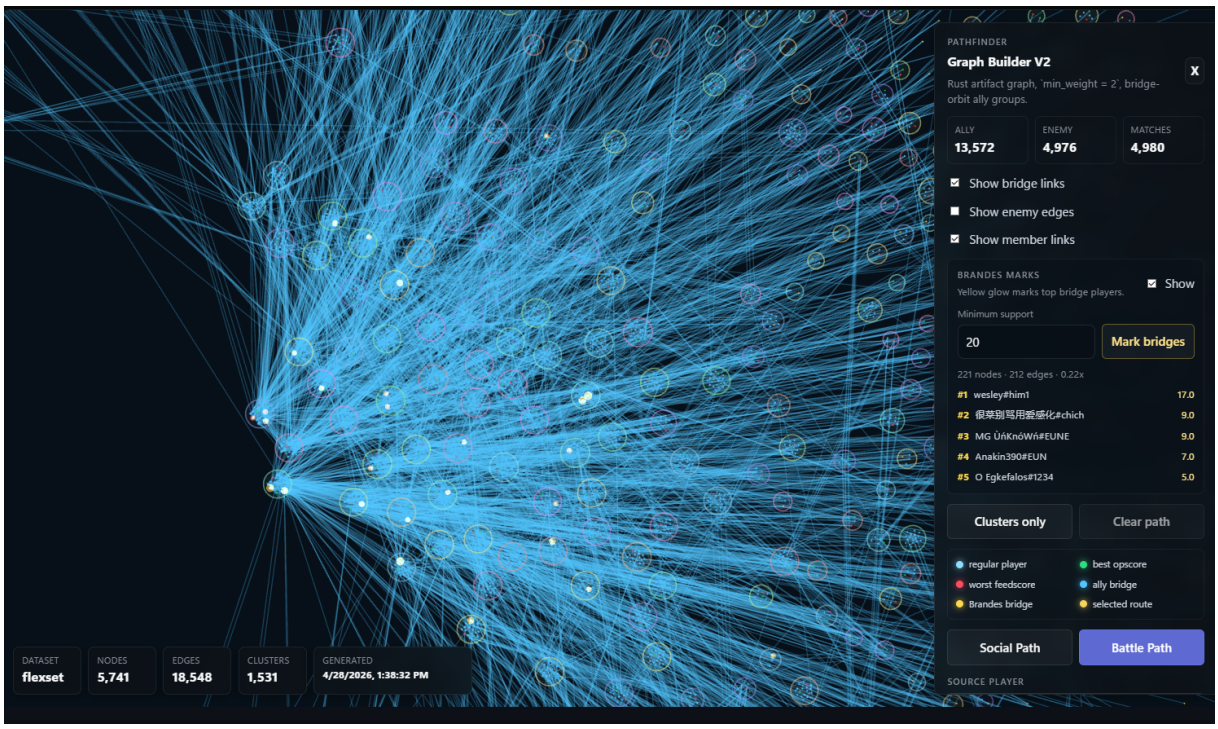


Figure 7.9: Broker players identified by betweenness centrality, highlighted in yellow in the graph.

8. The Database Sub-Project: PostgreSQL Schema and C# Browser

8.1 Background

The *docs/databaseproject* directory contains a formally designed relational database model built on PostgreSQL. This sub-project has dual value: it strengthens the academic database-design dimension of the thesis and provides a real, runnable tool for browsing analytic results.

8.2 The Formal Relational Schema

docs/databaseproject/schema.sql defines a complete PostgreSQL schema. The schema's ten main tables:

Table 8.1: Tables of the databaseproject PostgreSQL schema

Table	**Role in the schema**
players	Player identity, aggregated scores, role
matches	Match-level data: queue, patch, duration, winner
teams	The two teams of a match: side and objective data
match_participants	Participant data: champion, lane, kills, deaths, assists
player_relationships	Player–player ally/enemy connection with weight
clusters	Cluster metadata: type, algorithm
cluster_members	Membership records with role summaries
analysis_runs	Metadata for analysis runs
performance_snapshots	Player performance snapshots per run
path_queries	Pathfinding requests and results

The sections covering views, stored procedures, and triggers can be found in chapter 12 of the extended version [7].

8.3 The ASP.NET Core / C# Database Browser

The browser was built to browse the Premade Graph SQLite databases. On startup, the *ProjectImporter.Import()* process imports the players, clusters, cluster_members, and replay tables from the main system database. The browser reads the repository's *datasets.json* registry, and the desired dataset identifier can be passed as a URL parameter (*?dataset=flexset*, *?dataset=soloq*).

The browser operates with server-side HTML generation and no JavaScript frontend. The *Microsoft.Data.Sqlite* NuGet package provides database access. The application compiles and runs from the repository.

8.4 The Sub-Project's Place in the System

The databaseproject is not a paper exercise: *schema.sql* is runnable, and the C# browser compiles and runs. The SQLite-based main system is optimised for graph construction, analytics, and demo-pipeline serving; the PostgreSQL-based model aligns with the academic requirements of relational database design. The two serve different purposes within the same domain.

Part II

Tribal NeuroSim

9. Tribal NeuroSim: Foundations and Architecture

9.1 Objectives and Research Connection

Tribal NeuroSim is the direct continuation of the PremadeGraph graph-analysis pipeline. The PremadeGraph pipeline characterises each player cluster with a set of performance metrics: combat efficiency, resource management, objective control, risk tolerance, and team coordination. These values aggregate into cluster-level metrics (*A_combat*, *A_resource*, *A_map_objective*, *A_risk*, *A_team*), which form the initial genome sequence for tribes in Tribal NeuroSim.

Both dataset clusters appear simultaneously in the simulation. The research question: if cluster profiles derived from Flex Queue and SoloQ compete under identical simulation rules, does the underlying graph structure produce a measurable difference in the emerging survival strategies?

The experimental framework is an exploratory transfer study [10]. It does not attempt to directly predict real match outcomes and does not posit psychological or sociological causality.

9.2 Agent-Based and Multi-Agent Model

Tribal NeuroSim is an agent-based model (ABM): each tribe is an independent decision-making entity that acts solely on the basis of its own local state and immediate spatial neighbourhood. Macro-level patterns (expansion, wars, empire formation, extinction) emerge exclusively as the aggregate of individual decisions [10].

The system is also interpretable as a multi-agent system (MAS) [11]: each tribe operates as an independent agent with its own internal state, its own local perception, and its own decision logic (neural network). There is no direct communication between agents: interaction occurs exclusively through the shared spatial substrate.

The simulation exhibits the same structural phases: early expansion (ticks 0–300), consolidation (ticks 300–1000), then imperial wars (tick 1000+).

9.3 System Architecture

Figure 9.1 shows how responsibilities are divided between the three Tribal NeuroSim layers: the Rust engine (authoritative state), Node (intermediary), and MonoGame (display).

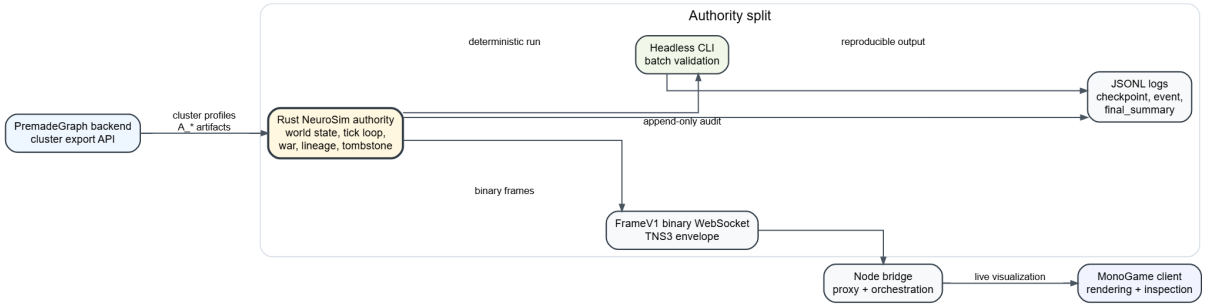


Figure 9.1: Tribal NeuroSim responsibility split: the Rust engine is the authoritative source of simulation state, while the Node and MonoGame layers act as intermediary and display.

The **Rust runtime** is the single source of authority: the tick cycle, world state, tribal neural decision-making, war and alliance resolution, fusion logic, tombstone entries, and event logging are all handled by the Rust binary [35]. The **C# MonoGame client** is exclusively for visualisation and operator interaction. The **Node layer** is a thin intermediary: a bridge between MonoGame and the Rust server.

This division is the direct lesson of the browser-based v2 prototype: when simulation state was coupled to the UI layer, ghost wars, memory leaks, and non-deterministic behaviour emerged.

The ghost-war issue specifically meant that completed wars and wars involving dead tribes never received a closed status: because no structured war-closing event existed, the system continued to track them as active, and their count accumulated in memory — in the final state of the early prototype, a single living tribe faced more than a hundred active ghost wars. In the current architecture every decision is made in the Rust binary, and a tribe's elimination atomically closes all its active wars (*WarCancelled*), so ghost wars cannot arise; the MonoGame client only renders the most recently received frame, and the simulation is reproducible from the same initial state at any point.

9.4 FrameV1 Binary Protocol

The Rust server and the MonoGame client communicate via binary WebSocket frames. Each frame begins with a 40-byte *TNS3* envelope (little-endian):

Table 9.1 presents the fields of the 40-byte header of the FrameV1 binary WebSocket frame.

Table 9.1: FrameV1 desktop envelope header.

Offset	Size	Field
0	4 B	Magic: *"TNS3"*
4	2 B	Version: *1*
6	2 B	HeaderBytes: *40*
8	2 B	PayloadKind: *2* (*FrameV1*)
10	2 B	Flags
12	4 B	PayloadBytes
16	8 B	Tick (*u64*)
24	4 B	Generation (*u32*)
28	4 B	RecordCount (number of living tribes)
32+	variable	V1 payload

The binary format decision is performance-driven: the Rust server handles the state of 599 tribes in 40 ms while the MonoGame client decodes and renders asynchronously. Avoiding JSON serialisation saves approximately 150–200 kB of unnecessary text overhead per tick in a 599-tribe world.

Details of event-driven observability and the determinism-bug fix (*clusters.sort_by(|a, b| a.id.cmp(&b.id))*) can be found in chapter 13 of the extended version [7].

10. Tribal NeuroSim: Game Logic and Neural Decision-Making

10.1 Neural Decision-Making and Cluster Priors

Every simulated tribe carries a neural network whose architecture follows the NEAT (*NeuroEvolution of Augmenting Topologies*) paradigm [23]. The system *does not apply backpropagation*: a tribe receives no training signal from real League of Legends matches. Instead, fitness is derived from the outcome of the simulation process — survival time, territory acquired, population size, and polity tier achieved — and is evaluated every thousand ticks [39].

At initialisation, each tribe receives cluster-level performance metrics derived from real match history as its initial genome sequence: **A_combat**, **A_resource**, **A_map_objective**, **A_risk**, **A_team**. During initialisation, each tribe is assigned a *Genome::new(11, 7)* structure with 11 input nodes, 7 output nodes, and 84 initial connections.

10.2 The 11 Input Signals and 7 Output Drives

Each tick, inference is run from the genome: the neural network reads 11 input signals and returns 7 output drives. Some inputs reflect the tribe's current state (food, population, danger); others are the performance profile inherited from the cluster, which mutates slowly at generational checkpoints. The outputs directly modulate the tribe's behavioural decisions: initiating war, expanding, forming alliances, or retreating.

Table 10.1 presents the 11 neural-network input signals, distinguishing dynamic (tick-level) signals from invariant (generational) signals.

Table 10.2 contains the seven output drives and their activation thresholds.

10.3 Fitness, Mutation, and Lineage Inheritance

Fitness consists of five components: *survival_ticks*, *territory*, *population*, *polity_tier*, and *war_record*. Mutation occurs every 1000 ticks. The base mutation rate is $\mu_0 = 0{,}05$; this is differentiated

Table 10.1: The 11 input signals of the neural network.

Index	Variable name	Description	Nature
0	*food_ratio*	Food stock / population	Dynamic
1	*pop_ratio*	Population / maximum capacity	Dynamic
2	*territory*	Claimed tiles / 100	Dynamic
3	*feed_risk*	Starvation proximity (1,0 = critical)	Dynamic
4	*a_combat*	Combat performance artifact	Invariant*
5	*a_resource*	Resource/economy artifact	Invariant*
6	*a_map_objective*	Objective control artifact	Invariant*
7	*a_team*	Team coordination artifact	Invariant*
8	*nearest_enemy*	Hex distance to nearest enemy	Dynamic
9	*nearest_ally*	Hex distance to nearest ally	Dynamic
10	*a_risk*	Risk tolerance artifact	Invariant*

* Updated only at generational checkpoints (every 1000 ticks) via mutation.

Table 10.2: The 7 output drives and their effect on tribe behaviour.

Variable name	Behavioural effect	Activation threshold
aggression	Modulates war-initiation probability	$> 0{,}45$
resource_drive	Territory expansion priority	$> 0{,}25$
goal_drive	Alliance initiation, polity advancement	$> 0{,}55$ / $> 0{,}50$
migration_drive	Triggers migration (from *Settling* state)	$> 0{,}55$
raid_drive	Opportunistic war against weaker neighbour	$> 0{,}35$
isolation	Rejection of alliance offers	$> 0{,}75$
expansion_speed	Tile-claim speed (cooldown modification)	Continuous

by fitness rank:

$$\mu_i = \begin{cases} 2\mu_0 & \text{if } r_i \leq 0{,}20 \text{ (bottom 20\%)} \\ \mu_0 & \text{if } 0{,}20 < r_i < 0{,}80 \\ \frac{1}{2}\mu_0 & \text{if } r_i \geq 0{,}80 \text{ (top 20\%)} \end{cases} \tag{10.1}$$

where $r_i \in [0, 1]$ is the relative fitness rank of tribe i in the population.

Fusion conditions for both parties: *goal_drive* $> 0{,}55$, neither is in an *AtWar* state, and *isolation* $< 0{,}75$. The heir genome from a fusion is a fitness-weighted blend: *Genome::inherit_from(parent_a, parent_b, fitness_a, fitness_b)*.

Output activity and fitness-differentiation data from the first experimental run (599 tribes, Flexset, 1200 ticks) and the liveness-fix details can be found in chapter 14 of the extended version [7]. Figure 10.1 shows the initial genome sequence built from cluster-level performance metrics.

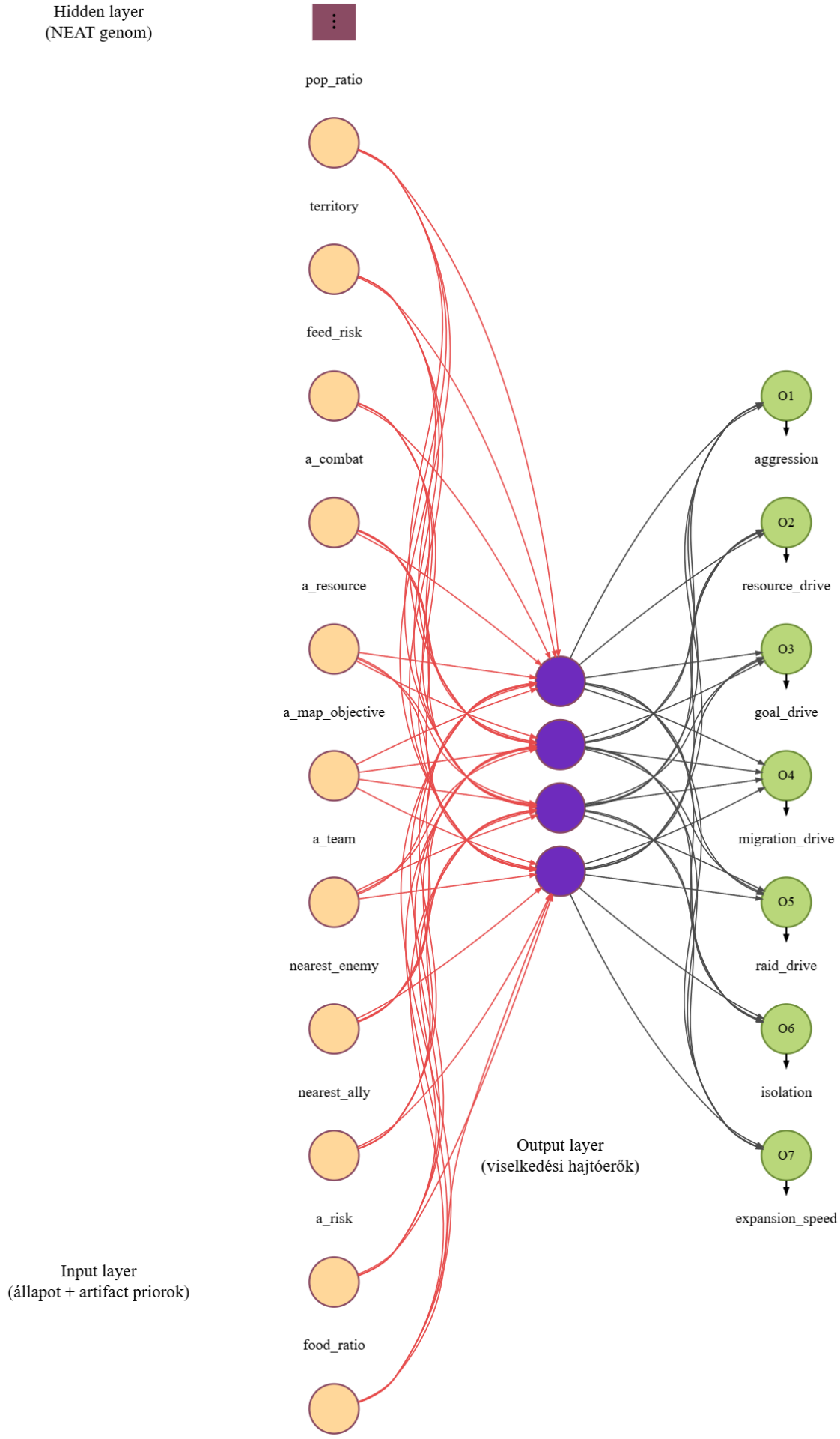


Figure 10.1: The genome graph: visualisation of the initial genome sequence built from performance metrics

11. Tribal NeuroSim: Experimental Runs and Results

11.1 World Mechanics

The simulation space is a hexagonal grid. Runs initialised from the flexset dataset take place on a map of approximately 36 100 tiles; soloq runs receive smaller worlds. Each tile is assigned a biome type (DenseForest, Riverland, DrySteppe, Mountains, Marsh, Cold) that determines its base food production rate.

Tribes evolve through five polity levels: *Tribe*, *City*, *Duchy*, *Kingdom*, and *Empire*. Each level-up requires meeting population, territory, and alliance thresholds; higher levels unlock greater maximum population and better combat efficiency, but also impose higher food-maintenance demands. Reaching Empire level is the victory condition: all four full runs and every run in the 103-seed sweep concluded with an Empire-level winner.

Combat outcomes are based on a Poisson process: the expected casualties are determined by the attacker's combat artifact and the contact zone (the number of adjacent tile pairs on the boundary between the two territories, $\lambda = a_{\text{combat}} \cdot n_{\text{contact}} \cdot 0{,}02$), but the concrete tick-level outcome is stochastic. The *liveness-fix* (14 May 2026) corrected three numerical errors — double normalisation of tribe statistics, oscillation of the migration drive, and an overly conservative expansion threshold — which had previously made the simulation static. Separate from this is the determinism-bug fix: unordered SQLite loading of clusters produced different tick orderings and therefore different outcomes; after introducing *clusters.sort_by(id)*, re-runs produce bit-identical results. Detailed advancement conditions and iterative design decisions can be found in the extended version [7].

11.2 Summary of the Four Runs

Summary of the four full experimental runs:

Table 11.1 summarises the main parameters and results of the four experimental runs.

Table 11.1: Comparison of the four final Tribal NeuroSim runs

Run	Dataset	Seed	Winner	Behaviour	Aggression	Migration	Final tick
run67fl7777	Flexset	7777	Tribe 355	Warband/Combat	0,18	0,88	2538
run67fl42	Flexset	42	Tribe 596	Warband/Combat	0,13	0,87	2487
run67sq42	SoloQ	42	Tribe 89	Vanguard/Risk	0,11	0,16	2046
run67sq7777	SoloQ	7777	Tribe 70	Supply/Resource	0,68	0,87	2541

Figure 11.1 shows the final state of the flexset run with seed=7777.

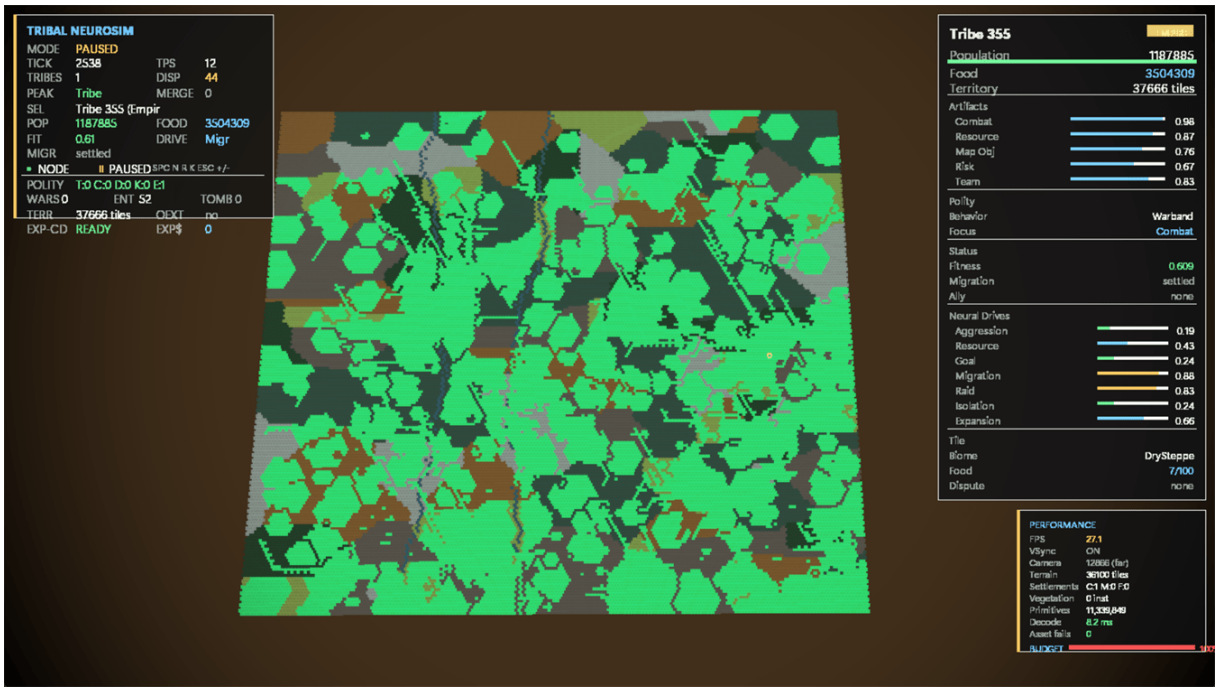


Figure 11.1: Flexset, seed=7777, tick=2538: Tribe 355 (Warband/Combat) victory, final territory 37 666 tiles.

Figure 11.2 shows the final state of the soloq run with seed=42, won by a tribe with a Vanguard/Risk profile.

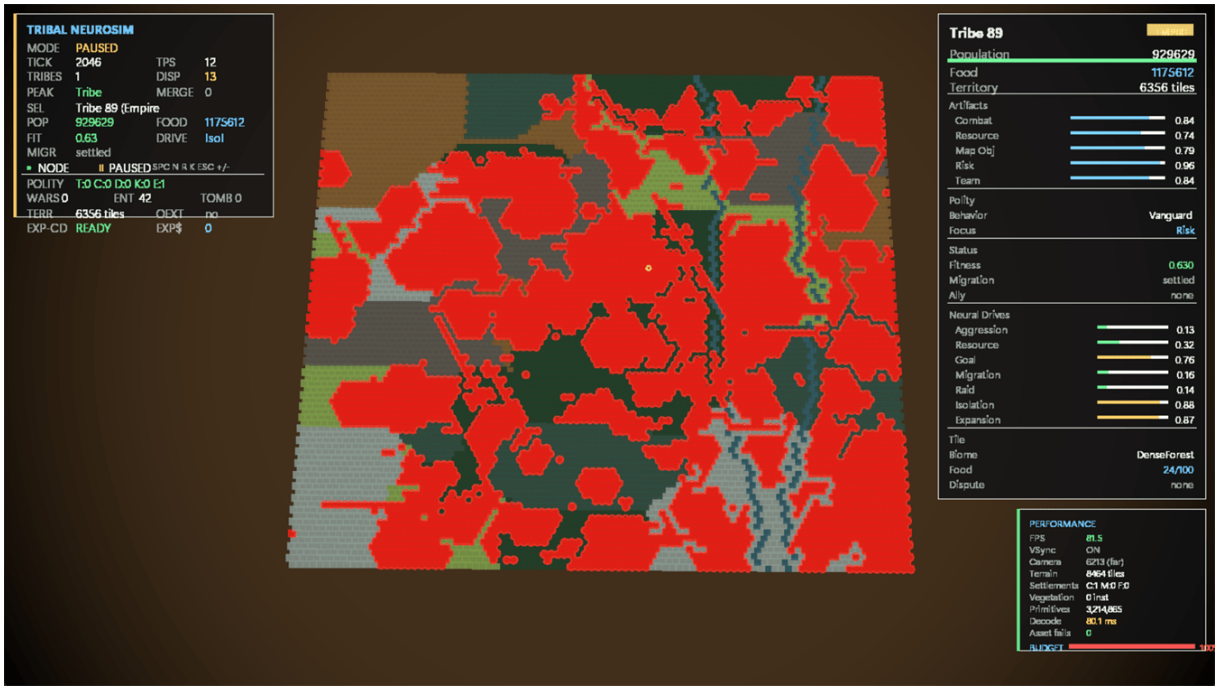


Figure 11.2: SoloQ, seed=42, tick=2046: Tribe 89 (Vanguard/Risk) victory, final territory 6 356 tiles.

Figure 11.3 shows the final state of the soloq run with seed=7777, won by a tribe with a Supply/Resource profile.

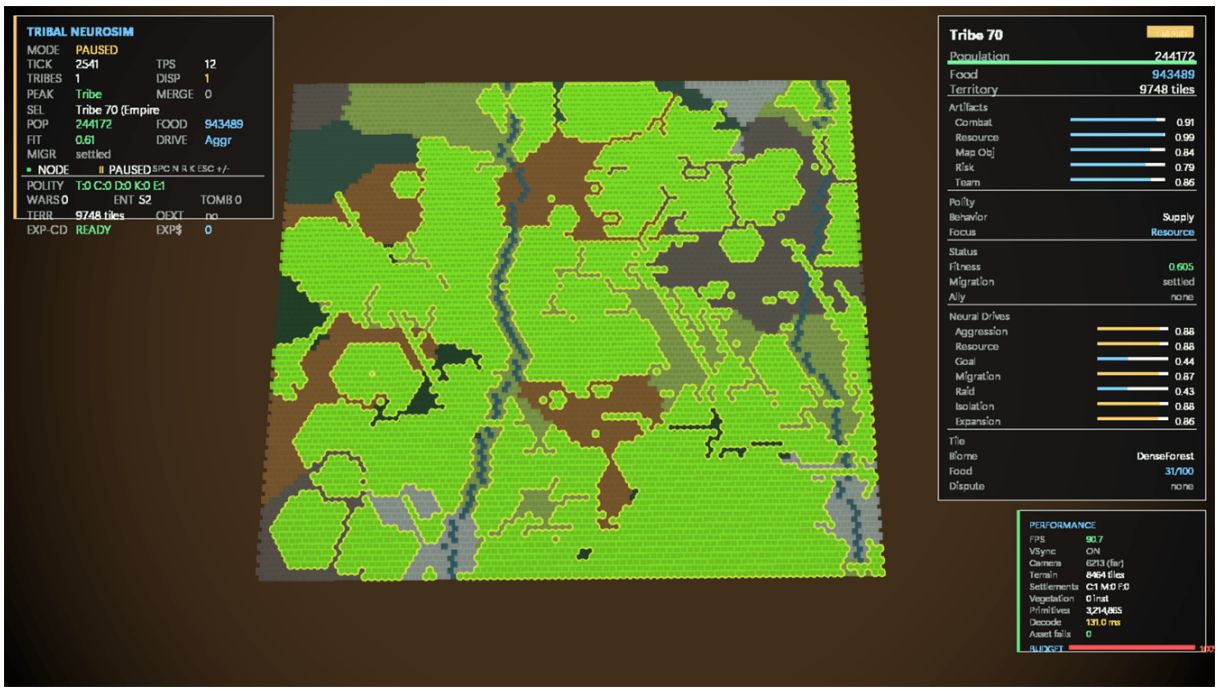


Figure 11.3: SoloQ, seed=7777, tick=2541: Tribe 70 (Supply/Resource) victory, final territory 9 748 tiles.

11.3 103-Seed Sweep and Winner Archetype Distribution

The four experimental runs are supplemented by a 103-seed soloq sweep. Every run in the sweep concluded with an Empire-level winner (103/103). Archetype distribution:

Table 11.2 contains the winning archetype distribution across the 103-seed soloq sweep.

Table 11.2: Winner archetype distribution across the soloq 103-seed sweep

Archetype	Runs	Share
Warband	36	35,0%
Supply	23	22,3%
Council	20	19,4%
Vanguard	14	13,6%
Pathfinders	10	9,7%
Total	103	100%

The average finishing tick is 2308,57; the average winner territory is 8 338 tiles. The Warband archetype leads with a slight plurality (35%) but does not hold strong, seed-independent dominance.

The log files of the 103-seed sweep contain approximately 35 000 lines of JSONL-format event logs per run. To aggregate the resulting data volume and identify the archetype distribution, an AI-based text analysis tool was used; the aggregated statistics were subsequently verified manually.

11.4 What the Results Show

Both tested flexset runs (seed=42 and seed=7777) ended in a Warband/Combat profile victory, with low aggression (0,13–0,18) and high migration (0,87–0,88). The soloq runs produced more variable winner profiles: seed=42 produced a Vanguard/Risk winner, seed=7777 a Supply/Resource winner.

This difference is interpretable from the underlying graph structure. The flexset dataset carries more organised, recurring connection patterns, while the soloq dataset captures more individual performance profiles without persistent co-queue structure.

The interpretation is correlational, not causal. Correctness was verified through deterministic batch validation: in five parallel headless runs of flexset seed=42, Tribe 596 (Warband/Combat) won every time, with bit-identical *final_summary* records.

Under identical rule conditions, *flexset* priors produced a Warband/Combat winner for both seeds (aggression 0,13–0,18), while *soloq* priors produced a different archetype per seed (Vanguard and Supply respectively) — the initial cluster prior measurably changes the outcome. The results do not demonstrate actual player psychology and do not predict outcomes of real League of Legends matches.

12. Conclusion

12.1 What the Thesis Presents

The thesis presents a system that starts from real League of Legends match data and from it builds player profiles, relationships, graphs, clusters, analytic results, and finally a simulation experiment. Beyond kill and death counts, the system records which players a given player ends up in a game with, who they face as opponents, what recurring relationships form around them, and what larger network patterns emerge from those.

If the essence of the project had to be reduced to a single point, it would be the existence of the full chain rather than any individual component. A Dijkstra implementation or a React page in isolation is a routine project. What makes the system interesting is the connectedness: from raw match data, through the graph, clusters, and Rust analytics, it reaches a reproducible simulation experiment, and every step is verifiable.

12.2 Main Results

The data-collection layer handles two different queue types in parallel. Keeping *flexset* and *soloq* separate is important because the two queues do not measure the same phenomenon: Flex Queue is better suited for examining recurring connection patterns; SoloQ serves as a control and an individual performance baseline.

The *opscore* and *feedscore* metrics provide role-aware scores. They are not perfect truth measurements, but they allow performance indicators to be used as inputs to graph analytics.

The *flexset* graph has a core-periphery-like structure: there are denser, more strongly connected core regions and many looser, smaller peripheral groups. This follows from the queue mechanics.

The assortativity analysis shows a positive association for *opscore* in social-path mode (0.235): connected players carry more similar performance profiles than would be expected under random connections.

The Rust-based algorithmic layer — pathfinding, assortativity, betweenness centrality — is deterministic and reproducible: the same input data always produces the same result.

12.3 The Role of Tribal NeuroSim

Tribal NeuroSim is a transfer experiment: it examines whether the performance profiles extracted from the graph and its clusters can be carried over into the initial parameters of an evolutionary, multi-agent simulation.

Based on the four runs, profiles from the Flex Queue showed more consistent behaviour: a similar Warband/Combat winning archetype emerged under both tested seeds. The SoloQ runs were more variable — different seeds produced different winning profiles. The flexset priors thus produced more consistent outcomes; the soloq priors produced more variable ones — under an identical simulation rule set.

12.4 Retrospective: What Worked Well

In retrospect, the best decision was not forcing everything into a single technology. Python stayed for rapid data collection and experimentation; Rust took over where performance and determinism were needed; Node/Express and React made the whole thing presentable. Forcing all of this into a single language would have made at least one layer suffer.

The project's strength is the coherent chain: from raw API data through normalised player profiles, graph construction, and clustering, all the way to analytic results — every step is verifiable and re-runnable. Assortativity, betweenness centrality, and the core-periphery interpretation are not just visualisation; they ask real questions about the network.

12.5 Limitations and Open Questions

A few limitations are worth keeping in mind when interpreting the results.

The dataset is not a random sample. The seed player selection, the premade-probe mechanism, and the queue type together determine which player population enters the graph. The resulting structure is a product of the given collection strategy, not a general description of all EUNE Master+ players.

opscore and feedscore are useful and interpretable metrics, but not external ground-truth measures. The role expectations and curve parameters are part of the model — different construction choices would yield different results.

Even in the graph, an enemy connection is not equivalent to genuine conflict. Repeated opposition can arise from matchmaking MMR bands, shared time-of-day, or the fact that the active Master+ Flex Queue player pool is inherently small. This is why the signed graph remained an experimental option.

Temporal consistency and community cohesion analyses fell out of the current scope. The excluded analyses are not uninteresting; the data and methodology did not, however, support defensible claims.

12.6 Closing Remarks

The end result of the project can be stated simply: it is possible to build a system from competitive game data that holds up from a software engineering perspective and can also pose research questions. The PremadeGraph data-collection, data-modelling, graph-analytic, and simulation chain demonstrates this.

The connectivity between steps was not planned in advance. Raw match data became normalised player profiles, profiles became a relationship graph, the graph became clusters and analytic metrics, and clusters became simulation priors — each step taking the previous step's output as its input.

As ABBA put it decades ago: *the winner takes it all*. In Tribal NeuroSim, this is not a metaphor but a direct consequence of the mechanics.

12.7 Where This Applies Elsewhere

The methodology is not tied to League of Legends. The Riot API was a convenient choice because it is publicly available, rich, and structured. But the same pipeline would run equally well on GitHub collaboration data, NBA performance statistics, or PubMed co-authorship networks — anywhere there is recurring interaction, measurable performance, and some form of group structure.

Graph clustering, for example, describes a corporate organisational network just as naturally as a player population: in both cases the question is which actual working groups emerge behind the formal structure. Betweenness centrality measures the same thing in infrastructure planning or epidemiology — which node's removal most disrupts the network.

The five Tribal NeuroSim artifacts are not LoL-specific concepts either. Combat, resource, map_objective, risk, team — these translate to other domains, and the evolutionary simulation framework can start from any cluster profile where group-level performance indicators can be extracted. The video game here was a tool, not a constraint.

Bibliography

[1] Riot Games, “Developer Portal Documentation,” Riot Games. [Online]. Available: https://developer.riotgames.com/docs/portal [Accessed: 28 June 2026.]

[2] Riot Games, “League of Legends API Documentation,” Riot Games. [Online]. Available: https://developer.riotgames.com/docs/lol [Accessed: 28 June 2026.]

[3] OP.GG, “Guide to Features,” OP.GG Help Center. [Online]. Available: https://help.op.gg/hc/en-us/sections/48325746929689-Guide-to-Features [Accessed: 1 July 2026.]

[4] SQLite Consortium, “SQLite Documentation,” SQLite.org. [Online]. Available: https://www.sqlite.org/docs.html

[5] West, B., “PyVis Network Documentation,” PyVis. [Online]. Available: https://pyvis.readthedocs.io/en/latest/documentation.html

[6] Hagberg, A., Schult, D., and Swart, P., “NetworkX: greedy_modularity_communities,” NetworkX Developers. [Online]. Available: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.modularity_max.greedy_modularity_communities.html

[7] T. Krisztián-Erik, *Premade Graph Explorer – extended version*, BSc thesis, George Emil Palade University of Medicine, Pharmacy, Science, and Technology, Târgu-Mureș, 2026. [Online]. Available: https://github.com/wpowertechgit/premadegraph/blob/main/docs/latex/main.pdf

[8] F. Heider, “Attitudes and Cognitive Organization,” *J. Psychology*, vol. 21, no. 1, pp. 107–112, 1946. doi:10.1080/00223980.1946.9917275

[9] D. Cartwright and F. Harary, “Structural Balance: A Generalization of Heider’s Theory,” *Psychological Rev.*, vol. 63, no. 5, pp. 277–293, 1956. doi:10.1037/h0046049

[10] E. Bonabeau, “Agent-based modeling: Methods and techniques for simulating human systems,” *Proc. Nat. Acad. Sci.*, vol. 99, suppl. 3, pp. 7280–7287, 2002. doi:10.1073/pnas.082080899

[11] M. Wooldridge and N. R. Jennings, “Intelligent Agents: Theory and Practice,” *Knowledge Eng. Rev.*, vol. 10, no. 2, pp. 115–152, 1995.

[12] M. E. J. Newman, “Mixing Patterns in Networks,” *Physical Rev. E*, vol. 67, no. 2, p. 026126, 2003. doi:10.1103/PhysRevE.67.026126

[13] M. Girvan and M. E. J. Newman, “Community Structure in Social and Biological Networks,” *Proc. Nat. Acad. Sci.*, vol. 99, no. 12, pp. 7821–7826, 2002. doi:10.1073/pnas.122653799

[14] S. Fortunato, “Community Detection in Graphs,” *Physics Reports*, vol. 486, no. 3–5, pp. 75–174, 2010. doi:10.1016/j.physrep.2009.11.002

[15] S. Fortunato and M. Barthelemy, “Resolution Limit in Community Detection,” *Proc. Nat. Acad. Sci.*, vol. 104, no. 1, pp. 36–41, 2007. doi:10.1073/pnas.0605965104

[16] A. Clauset, M. E. J. Newman, and C. Moore, “Finding Community Structure in Very Large Networks,” *Physical Rev. E*, vol. 70, no. 6, p. 066111, 2004. doi:10.1103/PhysRevE.70.066111

[17] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. doi:10.1007/BF01386390

[18] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968. doi:10.1109/TSSC.1968.300136

[19] A. V. Goldberg and C. Harrelson, “Computing the Shortest Path: A* Search Meets Graph Theory,” in *Proc. 16th Annu. ACM–SIAM Symp. Discrete Algorithms (SODA)*, Vancouver, Canada, 2005, pp. 156–165.

[20] U. Brandes, “A Faster Algorithm for Betweenness Centrality,” *J. Math. Sociology*, vol. 25, no. 2, pp. 163–177, 2001. doi:10.1080/0022250X.2001.9990249

[21] J. Maynard Smith and G. R. Price, “The Logic of Animal Conflict,” *Nature*, vol. 246, pp. 15–18, 1973. doi:10.1038/246015a0

[22] J. Maynard Smith, *Evolution and the Theory of Games*. Cambridge, U.K.: Cambridge Univ. Press, 1982. doi:10.1017/CBO9780511806292

[23] K. O. Stanley and R. Miikkulainen, “Evolving Neural Networks through Augmenting Topologies,” *Evol. Computation*, vol. 10, no. 2, pp. 99–127, 2002. doi:10.1162/106365602320169811

[24] G. Facchetti, G. Iacono, and C. Altafini, “Computing Global Structural Balance in Large-Scale Signed Social Networks,” *Proc. Nat. Acad. Sci.*, vol. 108, no. 52, pp. 20953–20958, 2011. doi:10.1073/pnas.1109521108

[25] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, “Ten Simple Rules for Reproducible Computational Research,” *PLoS Comput. Biology*, vol. 9, no. 10, p. e1003285, 2013. doi:10.1371/journal.pcbi.1003285

[26] T. Krisztián-Erik, “Premade Graph Repository,” GitHub. [Online]. Available: `https://github.com/wpowertechgit/premadegraph`

[27] F. Bahrololloomi, S. Sauer, M. Klonowski, R. Horst, and R. Dörner, “A Machine Learning Based Analysis of e-Sports Player Performances in League of Legends for Winning Prediction Based on Player Roles,” in *Proc. 16th Int. Joint Conf. Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP)*, Online, 2022.

[28] A. R. Novak, K. J. M. Bennett, M. A. Pluss, and J. Fransen, “Performance Analysis in Esports: Modelling Performance at the 2018 League of Legends World Championship,” *Int. J. Sports Sci. Coaching*, vol. 17, no. 2, pp. 315–325, 2022.

[29] R. Ghawi, J. Müller, and J. Pfeffer, “Team Communication Networks and Performance in Online Games,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, Glasgow, U.K., 2019.

[30] P. Sánchez-González and S. González-Bailón, “Network Structure and Team Performance in Professional League of Legends,” *J. Computational Social Sci.*, vol. 6, pp. 225–248, 2023. doi:10.1007/s42001-022-00182-4

[31] Microsoft, “ASP.NET Core Documentation,” Microsoft Docs. [Online]. Available: `https://docs.microsoft.com/en-us/aspnet/core/`

[32] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL.org. [Online]. Available: `https://www.postgresql.org/docs/`

[33] M. Fowler, “Event Sourcing,” Mar. 2005. [Online]. Available: `https://www.martinfowler.com/eaaDev/EventSourcing.html`

[34] T. Munzner, “A Nested Model for Visualization Design and Validation,” *IEEE Trans. Visualization Comput. Graphics*, vol. 15, no. 6, pp. 921–928, 2009.

[35] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “Safe Systems Programming in Rust,” *Commun. ACM*, vol. 64, no. 4, pp. 144–152, 2021.

[36] S. Klabnik and C. Nichols, *The Rust Programming Language*. San Francisco, CA, USA: No Starch Press, 2019.

[37] M. S. Granovetter, “The Strength of Weak Ties,” *Amer. J. Sociology*, vol. 78, no. 6, pp. 1360–1380, 1973. doi:10.1086/225469

[38] S. Milgram, “The Small World Problem,” *Psychology Today*, vol. 2, no. 1, pp. 60–67, 1967.

[39] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. Cambridge, MA, USA: MIT Press, 1992.

[40] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[41] J. Ziv and A. Lempel, “A Universal Algorithm for Sequential Data Compression,” *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, 1977. doi:10.1109/TIT.1977.1055714

[42] F. W. Lanchester, *Aircraft in Warfare: The Dawn of the Fourth Arm*. London, U.K.: Constable and Company Limited, 1916.

[43] J. Leskovec, D. Huttenlocher, and J. Kleinberg, "Signed Networks in Social Media," in *Proc. ACM SIGCHI Conf. Human Factors in Computing Systems (CHI)*, Atlanta, GA, USA, 2010, pp. 1361–1370. doi:10.1145/1753326.1753532

[44] J. Lerner, "Structural Balance in Signed Networks: Separating the Probability to Interact from the Tendency to Evaluate," *Social Networks*, vol. 45, pp. 22–43, 2016. doi:10.1016/j.socnet.2015.11.001

[45] M. Mora-Cantallops and M.-A. Sicilia, "MOBA Games: A Literature Review," *Entertainment Computing*, vol. 26, pp. 128–138, 2019. doi:10.1016/j.entcom.2018.02.005